DTSC 691 Capstone Project

Optimal Baseball Pitch Strategy

Erik Wagner

Introduction

Baseball has recently become an analytical goldmine, as showcased in the 2011 film Moneyball, where a team with limited money uses data analytics to identify undervalued players. The true creator of this era of analytical baseball is a man named Bill James, who authored more than 2 dozen books on the statistics of baseball, coining a term known as sabermetrics. His influence in baseball became so widespread that in 2006, Time Magazine named him as one of the 100 most influential people in the world, nearly 30 years after his first published book on the topic.

Now every team in professional baseball has entire divisions dedicated to analytics, who are all trying to identify the next trend that can give their team an advantage. This use of data science has become such a trend in fact that major league baseball actually has had to ban some of the strategies that data science has provided.

Goals of the project

With this being the true Golden Age of data science and machine learning in baseball, there is an opportunity to be a part of this in an oft overlooked side of the game. One of the most common machine learning projects that can be seen regarding baseball is a pitch predictor, but rarely, rarely is the other side of the same situation ever seen.

The aim of this project will be to answer the question “What is the optimal strategy for a baseball pitcher in any given situation?” As a secondary goal, clustering will be used to classify pitch type, which will be vital in achieving an optimal strategy.

The aim of the project can more simply be seen as finding the pitch type, horizontal location, and vertical location that will minimize a batter’s opportunity for a successful at-bat in any given situation.

Luckily, Major League Baseball has set up 12 ultra high-speed cameras at each baseball stadium, which are used to create a massive amount of data that the MLB releases every night, commonly referred to as Statcast. This data includes common baseball statistics that are tracked, along with much more detailed pitch-by-pitch analysis, which is what this project will primarily be utilizing. Through various machine learning techniques, including

clustering and regression this project will be able to identify a given pitcher's arsenal (what types of pitches he throws), identify tendencies of both hitters and pitchers in various states, and ultimately use those to identify the best strategies for a pitcher.

The idea of an optimal pitch will depend on many factors. But, ultimately, the optimal strategies for a pitcher will be decided by two main factors; the pitch type and the location of the pitch. These factors are important independently, but generally will have to work jointly together.

In [57]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from pybaseball import statcast
from pybaseball import pitching_stats
from pybaseball import batting_stats
from pybaseball import statcast_batter
from pybaseball import batting_stats_bref
from pybaseball import statcast_pitcher
from pybaseball.statcast_pitcher_spin import statcast_pitcher_spin
from pybaseball import playerid_lookup
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import AgglomerativeClustering
from sklearn.mixture import GaussianMixture
from sklearn.cluster import DBSCAN
from sklearn.manifold import TSNE
from keras.preprocessing import sequence
from keras.models import Sequential
from keras.layers import Dense, Embedding
from keras.optimizers import Adam
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import LabelEncoder, OrdinalEncoder
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
import xgboost as xgb
from sklearn.metrics import accuracy_score, precision_score, recall_score, mean_squ
import sys
from sklearn.linear_model import LinearRegression, Lasso
from catboost import CatBoostRegressor
import seaborn as sns
from sklearn.metrics import silhouette_score
plt.style.use('ggplot')
pd.options.display.max_columns = 100
pd.options.display.min_rows = 100
```

Data Description

For this project, I will use pybaseball, which is an open-source package that has scraped data from three of the largest baseball data websites, FanGraphs, Baseball Reference, and Baseball Savant. This package will contain data on every pitch for every game since 2017, which is what I will be utilizing as my data source.

The data gathered is both on a broad scale of major statistics for each player, down to exact information on each specific pitch thrown. While all data has to be considered, the primary data statistics will be include:

In [2]:
```
'''Use pybaseball to import data. Statcast_pitcher/batter will be used for the main
while spin_data is going to be used for clustering pitches'''

statcast_data_pitcher = statcast_pitcher('2018-07-01', '2022-10-31', player_id = 47
statcast_data_batter = statcast_batter('2018-07-01', '2022-10-31', player_id = 4522
spin_data = statcast_pitcher_spin('2019-07-01', '2019-09-30', player_id = 477132)
```

```
Gathering Player Data
Gathering Player Data
Gathering Player Data
```

In [3]:
```
#check shape of first df
statcast_data_pitcher.shape
```

Out[3]:
```
(10013, 92)
```

In [4]:
```
statcast_data_pitcher.head()
```

Out[4]:

	pitch_type	game_date	release_speed	release_pos_x	release_pos_z	player_name	batte
0	FF	2022-10-12	92.5	1.56	6.03	Kershaw, Clayton	592518
1	FF	2022-10-12	92.5	1.35	6.04	Kershaw, Clayton	665742
2	SL	2022-10-12	88.0	1.48	6.06	Kershaw, Clayton	665742
3	FF	2022-10-12	91.6	1.49	6.01	Kershaw, Clayton	665742
4	FF	2022-10-12	92.0	1.46	6.06	Kershaw, Clayton	665742

In [5]:
```
#drop some of the very obviously unneeded columns
exploration_data_pitcher = statcast_data_pitcher.copy()
exploration_data_pitcher.drop(columns=['game_date', 'spin_rate_deprecated','break_a
                                       'des', 'game_type','home_team','away_team','t
                                       'tfs_zulu_deprecated','fielder_2','fielder_2.
                                       'fielder_6','fielder_7','fielder_7','fielder_
exploration_data_pitcher.shape
```

Out[5]: (10013, 70)

In [6]:
```
#checking for any particular insights
exploration_data_pitcher.describe()
```

Out[6]:

	release_speed	release_pos_x	release_pos_z	batter	pitcher	zone	h
count	9710.000000	9710.000000	9710.000000	10013.000000	10013.0	9710.000000	2
mean	86.458054	1.369742	6.260602	579275.787277	477132.0	9.091658	
std	5.993481	0.241831	0.138152	70250.152861	0.0	4.156908	
min	69.800000	-5.230000	0.110000	405395.000000	477132.0	1.000000	
25%	86.300000	1.220000	6.170000	542303.000000	477132.0	5.000000	
50%	88.300000	1.390000	6.260000	592885.000000	477132.0	9.000000	
75%	90.500000	1.540000	6.350000	641796.000000	477132.0	13.000000	
max	93.700000	3.330000	6.660000	683021.000000	477132.0	14.000000	

In [7]:
```
#check to see where there are na values
exploration_data_pitcher.isna().sum()
```

Out[7]:

pitch_type	303
release_speed	303
release_pos_x	303
release_pos_z	303
player_name	0
batter	0
pitcher	0
events	7281
description	0
zone	303
stand	0
p_throws	0
hit_location	7523
bb_type	8167
balls	0
strikes	0
game_year	0
pfx_x	303
pfx_z	303
plate_x	303
plate_z	303
on_3b	9412
on_2b	8531
on_1b	7547
outs_when_up	0
inning	0
hc_x	8172
hc_y	8172
sv_id	10013
vx0	303
...	
effective_speed	208
release_spin_rate	417
release_extension	390
game_pk	0
pitcher.1	0
fielder_8	0
release_pos_y	303
estimated_ba_using_speedangle	8274
estimated_woba_using_speedangle	8274
woba_value	7281
woba_denom	7442
babip_value	7281
iso_value	7281
launch_speed_angle	8274
at_bat_number	0
pitch_number	0
pitch_name	303
home_score	0
away_score	0
bat_score	0
fld_score	0
post_away_score	0
post_home_score	0
post_bat_score	0
post_fld_score	0

```
if_fielding_alignment              207
of_fielding_alignment              207
spin_axis                          393
delta_home_win_exp                   0
delta_run_exp                      246
Length: 70, dtype: int64
```

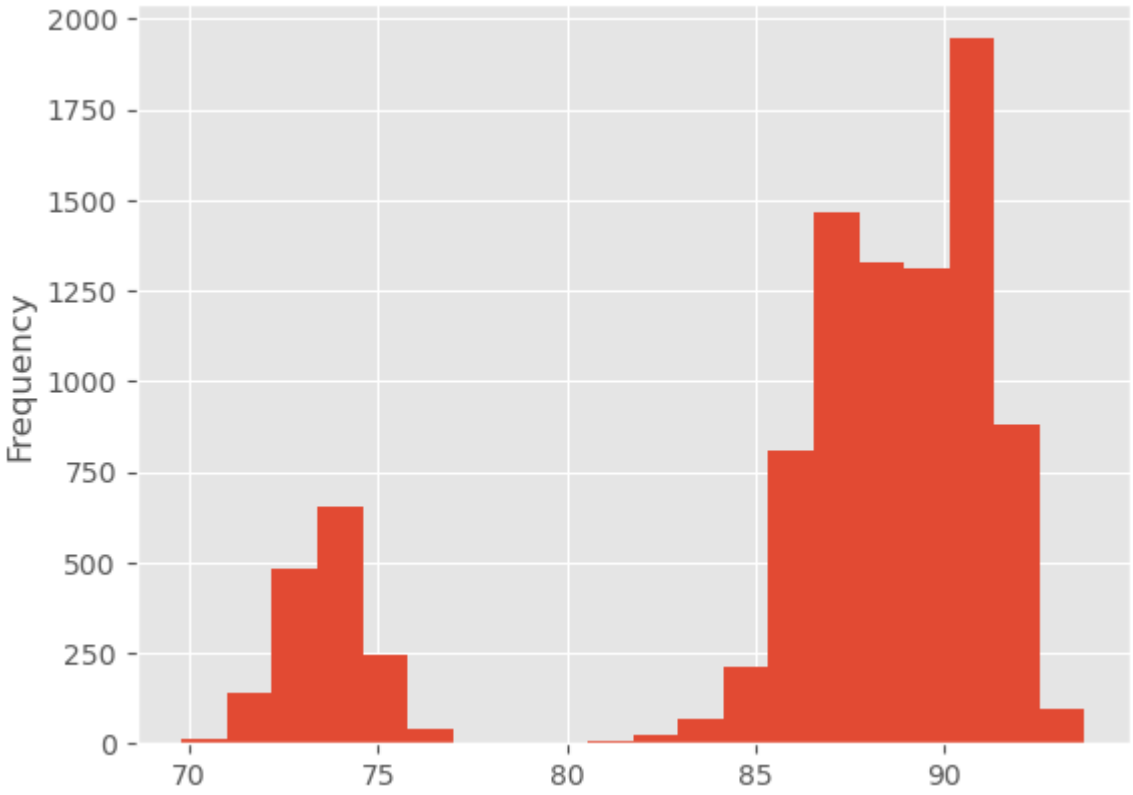
In [8]:
```
#drop some additional columns with many missing values, but not on_3b,on_2b, or on_
exploration_data_pitcher.drop(columns=['hit_location', 'bb_type','hc_x', 'hc_y','es
                                       'estimated_woba_using_speedangle', 'woba_valu
                                        'iso_value','launch_speed_angle'], inplace=T
```

In [9]:
```
#check for most common pitch thrown
exploration_data_pitcher['pitch_type'].value_counts()
```

Out[9]:
```
SL    4120
FF    3955
CU    1577
CH      44
SI      14
Name: pitch_type, dtype: int64
```

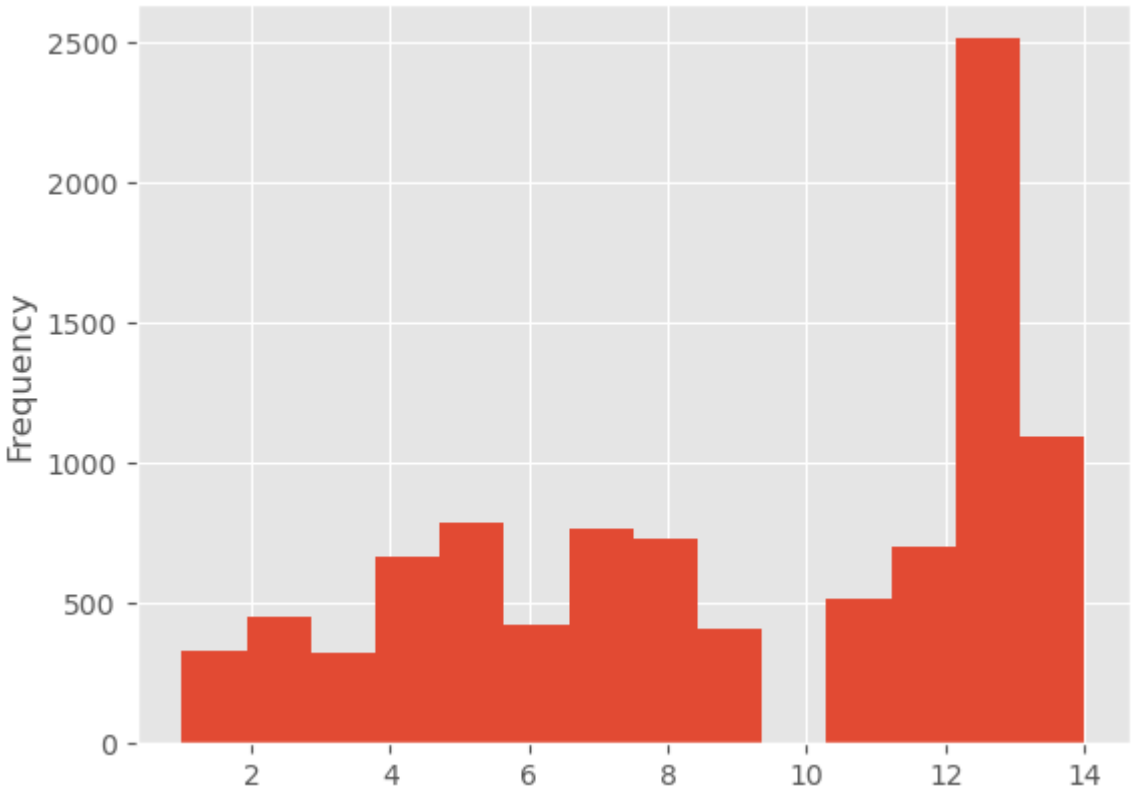
In [10]:
```
#check frequencies of different velocities
exploration_data_pitcher['release_speed'].plot(kind='hist',bins = 20)
```

Out[10]: <AxesSubplot:ylabel='Frequency'>



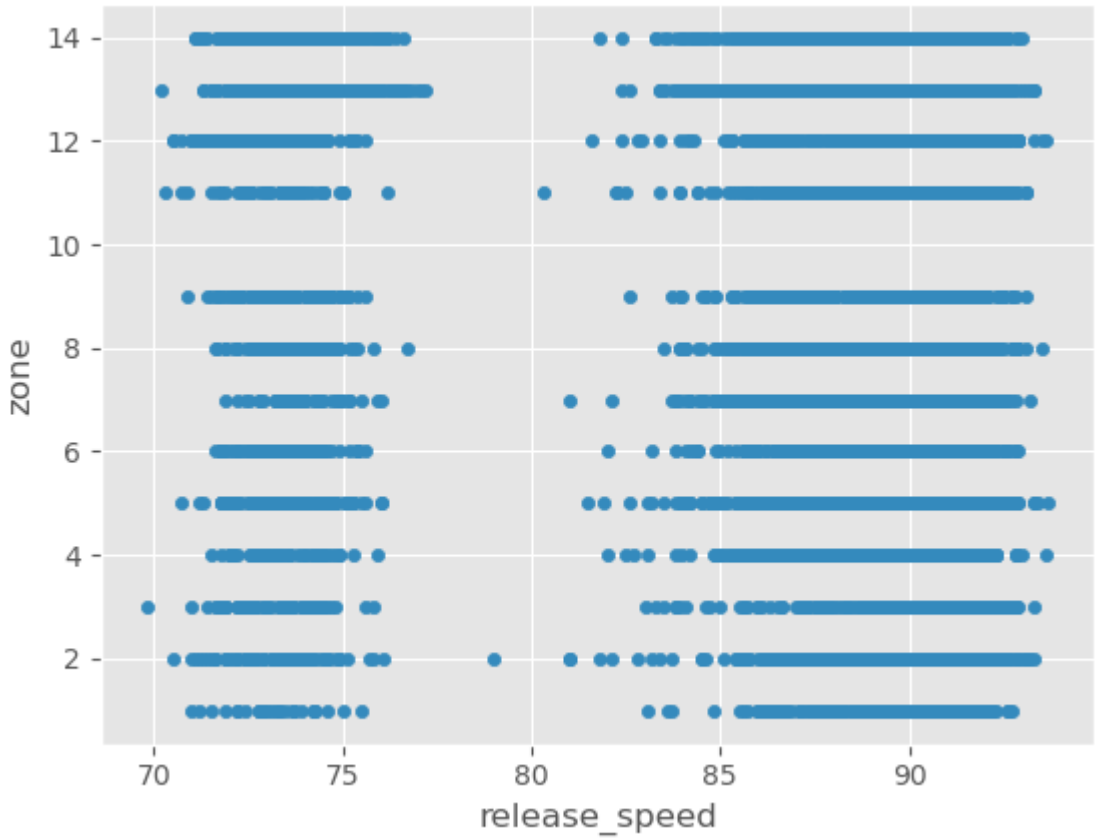
In [11]:
```
#check frequency that each zone is thrown to
exploration_data_pitcher['zone'].plot(kind='hist',bins=14)
```

Out[11]: <AxesSubplot:ylabel='Frequency'>



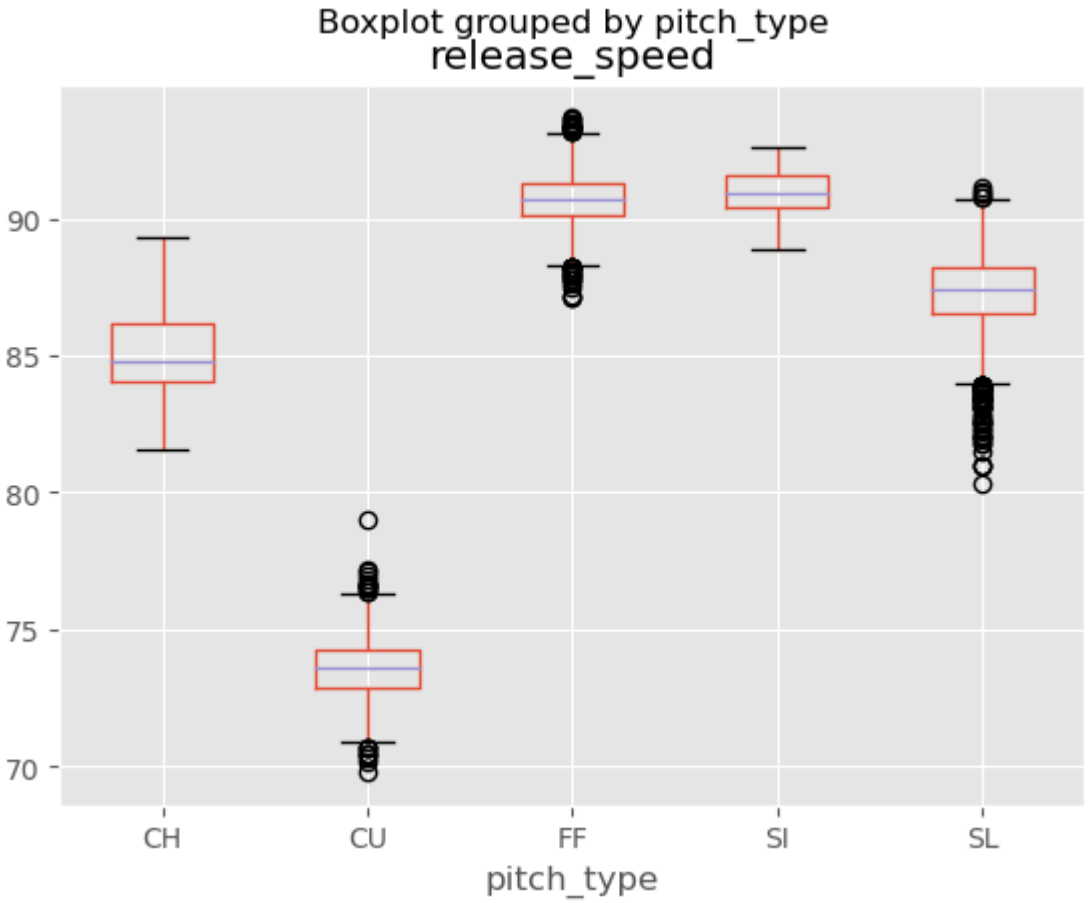
In [12]:
```
#sort of mimics a histogram above
exploration_data_pitcher.plot(kind='scatter', x='release_speed',y='zone')
```

Out[12]: <AxesSubplot:xlabel='release_speed', ylabel='zone'>



In [13]:
```
#box plot showing how velocity differs by pitch type
exploration_data_pitcher.boxplot(column='release_speed', by='pitch_type')
```

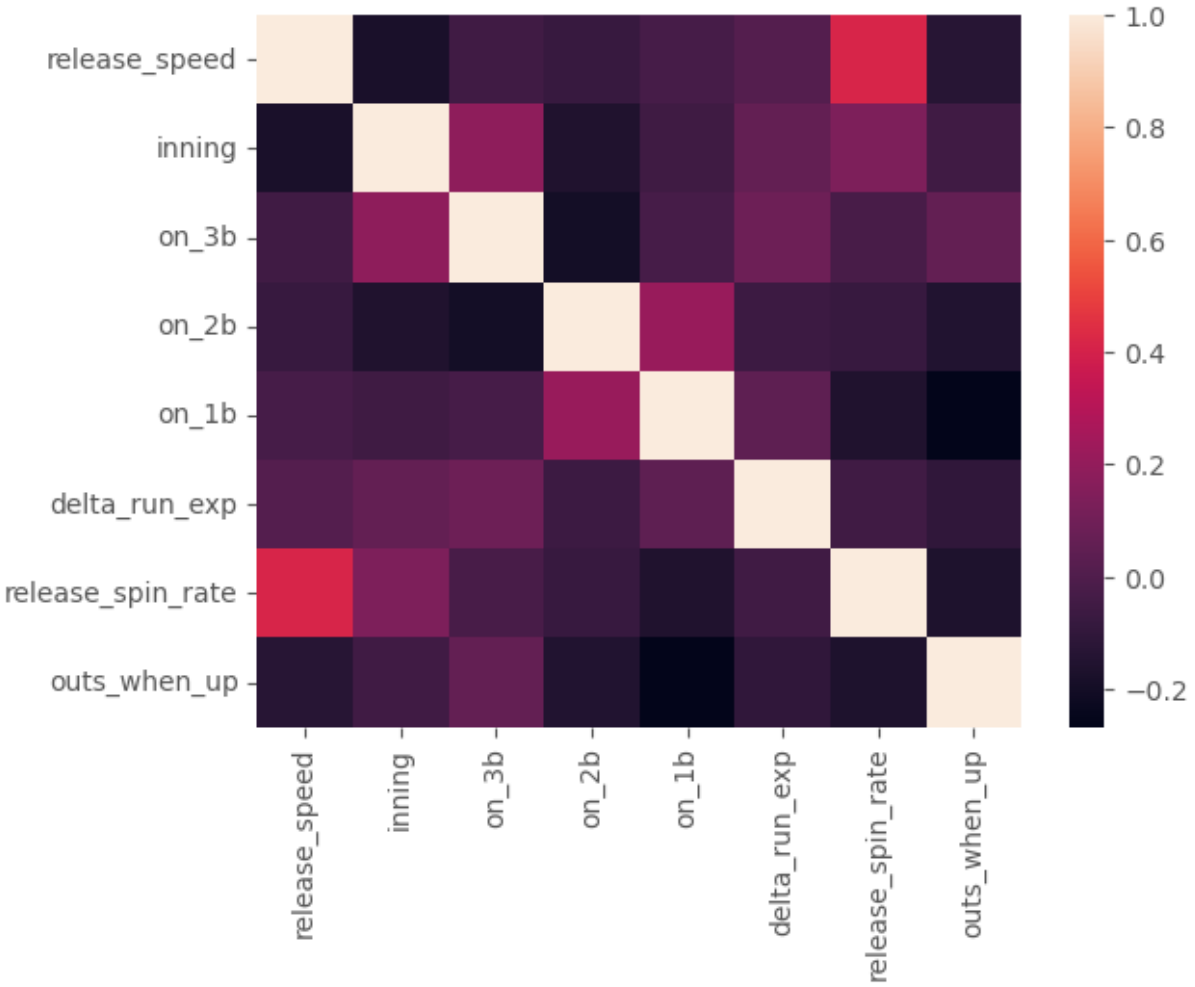
Out[13]:
```
<AxesSubplot:title={'center':'release_speed'}, xlabel='pitch_type'>
```



In [14]:
```
corr = exploration_data_pitcher[['release_speed','inning','on_3b','on_2b','on_1b','
```

In [15]:
```
#use correlation above to create heatmap, spin_rate and speed seem correlated
sns.heatmap(corr)
```

Out[15]:
```
<AxesSubplot:>
```



In [16]:
```
#here I am approaching this very similarly to the pitcher data because it is the sa
statcast_data_batter.shape
```

Out[16]: (2163, 92)

In [17]:
```
exploration_data_batter = statcast_data_batter.copy()
exploration_data_batter.drop(columns=['game_date', 'spin_rate_deprecated','break_an
                                      'des', 'game_type','home_team','away_team','t
                                      'tfs_zulu_deprecated','fielder_2','fielder_2.
                                      'fielder_6','fielder_7','fielder_7','fielder_
exploration_data_batter.shape
```

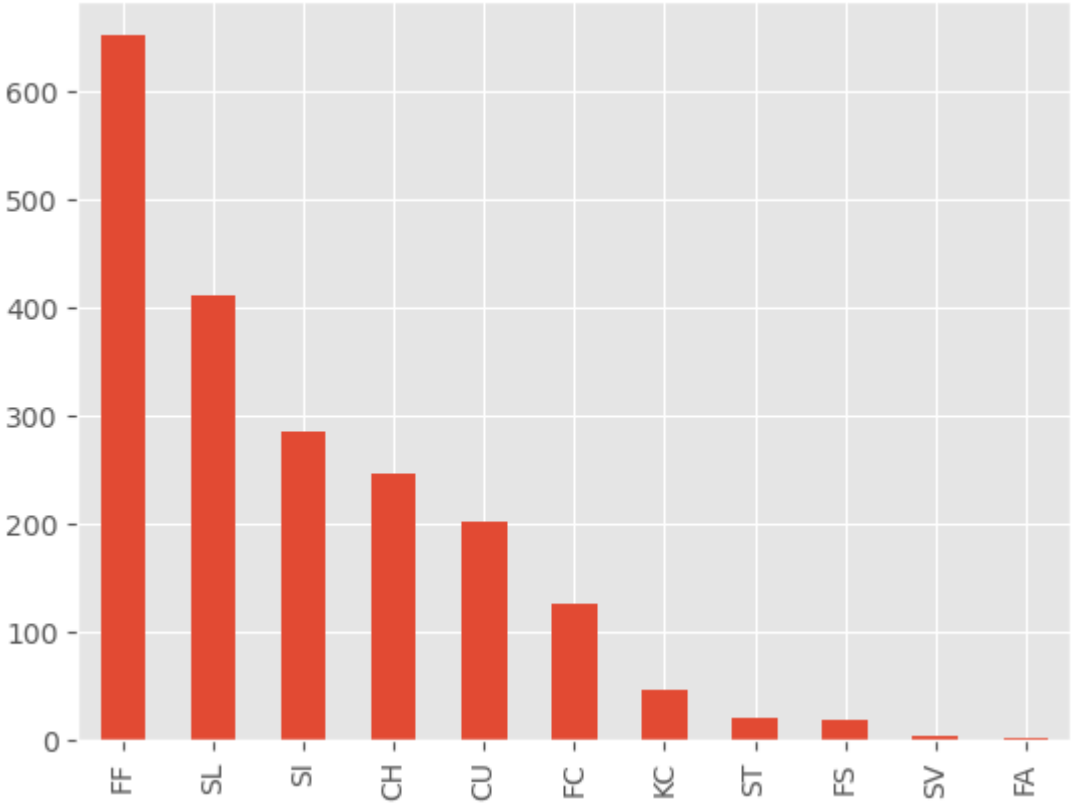
Out[17]: (2163, 70)

In [18]:
```
exploration_data_batter['pitch_type'].value_counts()
```

Out[18]:
```
FF    652
SL    411
SI    286
CH    246
CU    202
FC    127
KC     47
ST     20
FS     18
SV      4
FA      2
Name: pitch_type, dtype: int64
```

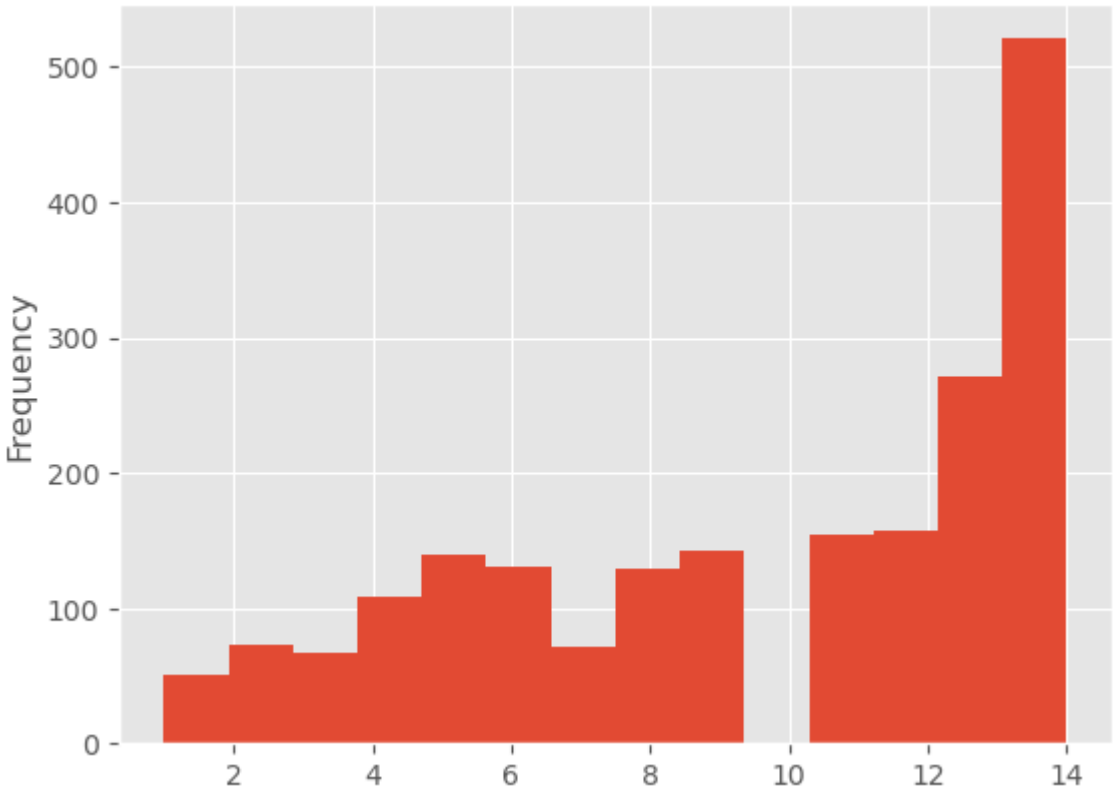
In [19]:
```
exploration_data_batter['pitch_type'].value_counts().plot(kind='bar')
```

Out[19]:
```
<AxesSubplot:>
```



In [20]:
```
exploration_data_batter['zone'].plot(kind='hist',bins=14)
```

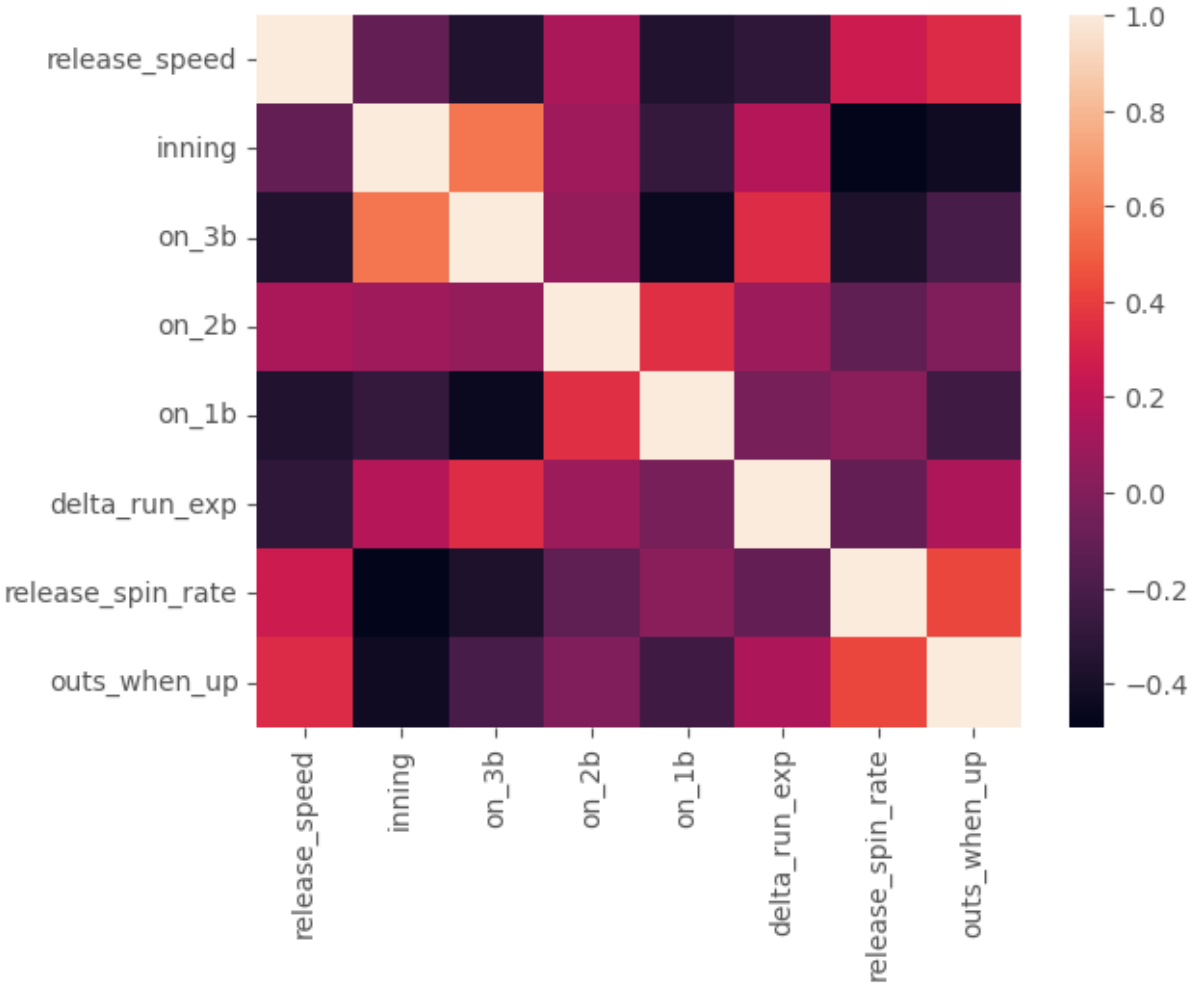
Out[20]:
```
<AxesSubplot:ylabel='Frequency'>
```



In [21]:
```
corr = exploration_data_batter[['release_speed','inning','on_3b','on_2b','on_1b','d
```

In [22]:
```
sns.heatmap(corr)
```

Out[22]: <AxesSubplot:>



In [23]:
```
#check spin_data
spin_data.head()
```

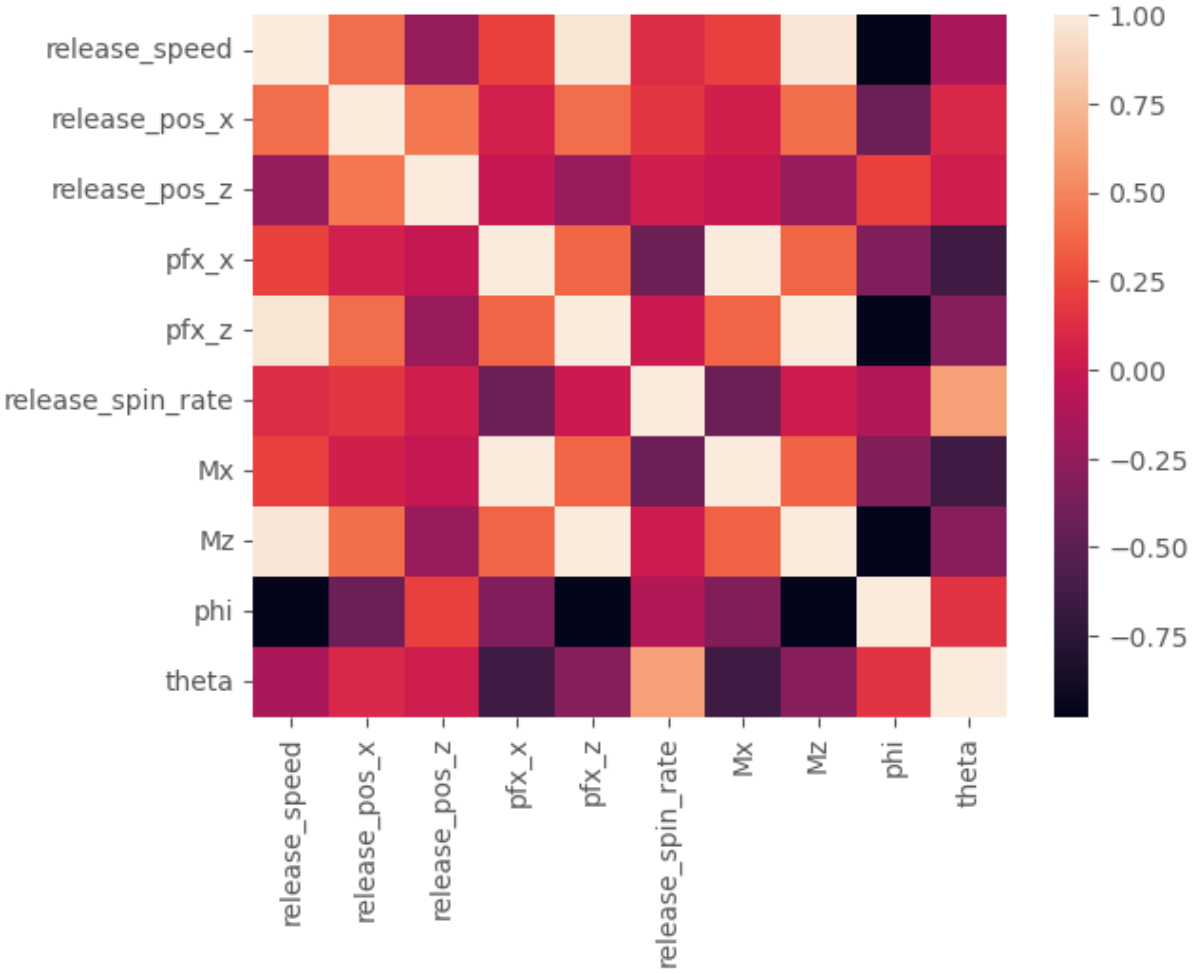
Out[23]:

	pitch_type	game_date	release_speed	release_pos_x	release_pos_z	player_name	batte
0	FF	2019-09-29	90.5	1.35	6.19	Kershaw, Clayton	518516
1	FF	2019-09-29	91.1	1.38	6.20	Kershaw, Clayton	518516
2	FF	2019-09-29	89.7	1.42	6.23	Kershaw, Clayton	518516
3	FF	2019-09-29	90.3	1.35	6.23	Kershaw, Clayton	518516
4	FF	2019-09-29	90.1	1.48	6.34	Kershaw, Clayton	518516

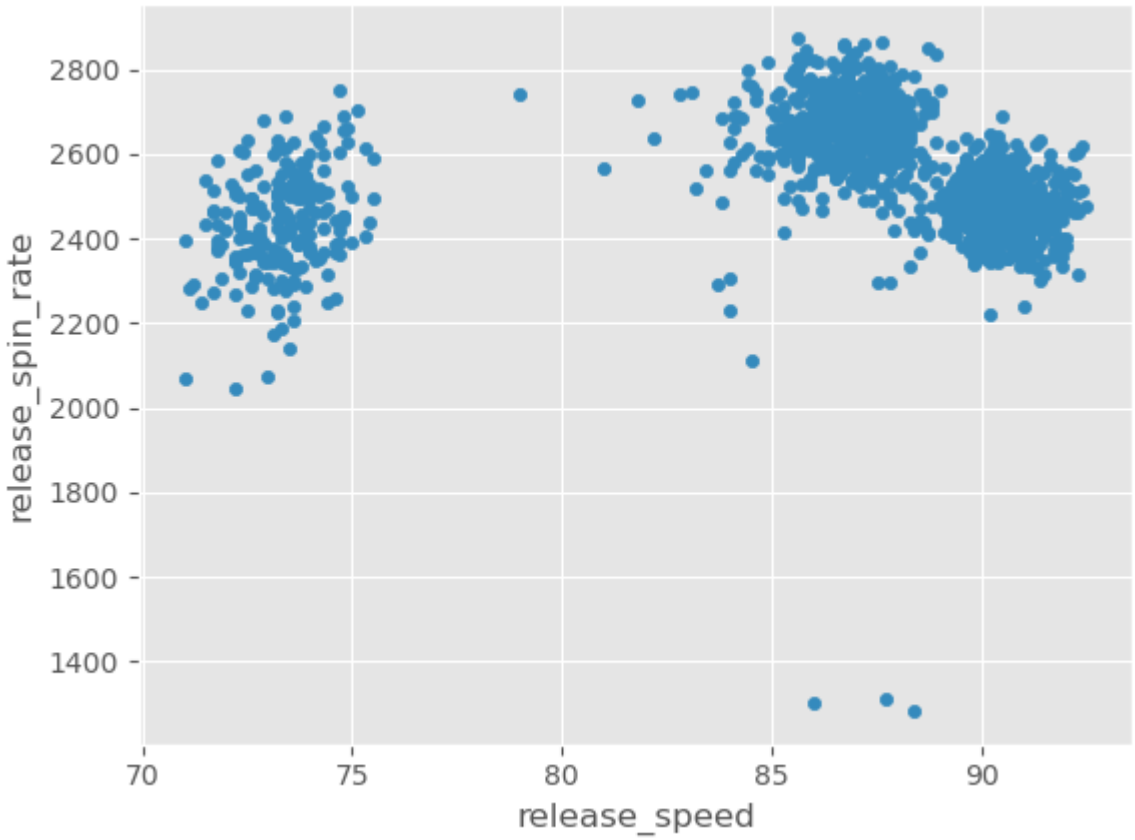
In [24]:
```
#correlation, then heatmap
corr1 = spin_data[['release_speed','release_pos_x','release_pos_z','pfx_x','pfx_z',
```

In [25]:
```
#heatmap seeing lots of correlations
sns.heatmap(corr1)
```

Out[25]:
```
<AxesSubplot:>
```



In [26]:
```
#checking correlation between spin_rate and release_speed
spin_data.plot(kind='scatter',x='release_speed',y='release_spin_rate')
plt.show()
```



In [27]:
```
datacluster = spin_data[['release_speed', 'release_spin_rate','pfx_x','pfx_z', 'spi
datacluster1 = datacluster.dropna()
datacluster1.mask(datacluster1.astype(object).eq('None')).dropna()
datacluster1 = datacluster1.loc[datacluster1['spin_axis'] > 10]
datacluster1
```

Out[27]:

	release_speed	release_spin_rate	pfx_x	pfx_z	spin_axis	effective_speed
0	90.5	2492.0	-0.45	1.68	195	89.9
1	91.1	2420.0	-0.12	1.61	184	91.1
2	89.7	2525.0	-0.14	1.42	186	89.2
3	90.3	2424.0	-0.32	1.80	190	89.3
4	90.1	2353.0	-0.32	1.51	192	90.2
5	90.7	2482.0	-0.28	1.50	191	91.1
6	90.8	2491.0	-0.01	1.76	180	90.2
7	86.9	2675.0	-0.39	0.46	220	86.6
8	86.5	2728.0	-0.65	0.77	220	85.4
9	85.7	2683.0	-0.57	0.49	229	84.5
10	74.3	2367.0	-0.06	-1.17	357	73.5
11	72.7	2399.0	-0.19	-1.52	353	71.6
12	90.0	2486.0	-0.03	1.75	181	89.2
13	90.2	2562.0	-0.15	1.70	185	89.8
14	87.0	2553.0	0.03	0.97	178	86.6
15	87.2	2677.0	0.09	0.63	172	86.4
16	90.7	2429.0	0.09	1.51	177	90.7
17	91.5	2485.0	-0.11	1.57	184	91.0
18	87.2	2571.0	-0.41	0.61	214	86.4
19	74.4	2471.0	-0.11	-1.58	356	73.5
20	91.1	2394.0	0.13	1.76	176	90.8
21	86.6	2575.0	-0.43	0.55	218	85.2
22	91.1	2353.0	-0.08	1.48	183	91.0
23	88.0	2589.0	-0.26	0.72	200	86.8
24	75.5	2495.0	0.00	-1.37	360	74.7
25	90.6	2531.0	-0.21	1.57	188	89.9
26	87.4	2568.0	0.09	0.88	174	86.7
27	91.0	2396.0	-0.09	1.58	183	90.5
28	86.9	2648.0	-0.33	0.55	210	86.2
29	90.0	2387.0	-0.09	1.68	183	90.5

	release_speed	release_spin_rate	pfx_x	pfx_z	spin_axis	effective_speed
...	...	...	...	...	...	...
1318	88.5	2742.0	-0.26	0.96	195	88.4
1319	91.6	2461.0	-0.17	1.62	186	91.4
1320	75.3	2613.0	-0.33	-1.45	347	74.9
1321	90.9	2448.0	-0.37	1.64	193	90.9
1322	91.5	2488.0	-0.25	1.64	189	91.8
1323	90.1	2533.0	-0.39	1.52	194	90.6
1324	89.6	2479.0	-0.32	1.64	191	89.1
1325	87.6	2651.0	-0.46	0.53	221	87.0
1326	91.3	2517.0	-0.15	1.64	185	91.0
1327	91.7	2478.0	-0.19	1.70	186	91.6
1328	91.4	2553.0	-0.02	1.72	181	91.2
1329	86.3	2717.0	-0.56	0.28	243	85.6
1330	86.9	2838.0	-0.52	0.26	243	85.9
1331	87.3	2817.0	-0.68	0.40	239	86.5
1332	73.8	2435.0	-0.30	-1.46	349	73.0
1333	90.8	2411.0	-0.46	1.22	200	90.7
1334	91.9	2402.0	-0.27	1.53	190	92.1
1335	75.4	2440.0	-0.16	-1.27	353	74.8
1336	87.7	2626.0	-0.48	0.57	220	87.2
1337	87.8	2608.0	-0.44	0.58	217	87.2
1338	88.5	2670.0	-0.42	0.83	207	87.9
1339	72.4	2363.0	-0.46	-1.33	341	72.0
1340	90.4	2403.0	-0.14	1.69	185	90.2
1341	88.1	2650.0	-0.48	0.83	210	87.9
1342	90.8	2489.0	-0.23	1.66	188	90.6
1343	91.7	2433.0	-0.30	1.74	190	91.6
1344	91.5	2437.0	-0.03	1.55	181	91.5
1345	90.9	2502.0	-0.05	1.70	182	90.5
1347	91.2	2452.0	-0.06	1.70	182	91.0

	release_speed	release_spin_rate	pfx_x	pfx_z	spin_axis	effective_speed
1348	90.3	2456.0	-0.28	1.61	190	89.9

1321 rows × 6 columns

In [28]:
```
scaler = StandardScaler()
scaledcluster = scaler.fit_transform(datacluster1)
scaledcluster = pd.DataFrame(scaledcluster)
scaledcluster
```

Out[28]:

	0	1	2	3	4	5
0	0.693818	-0.332537	-0.911751	0.856529	-0.430489	0.637623
1	0.796123	-0.840474	0.502545	0.787521	-0.622958	0.834995
2	0.557411	-0.099732	0.416830	0.600214	-0.587964	0.522490
3	0.659716	-0.812255	-0.354604	0.974829	-0.517975	0.538937
4	0.625615	-1.313137	-0.354604	0.688938	-0.482981	0.686966
5	0.727920	-0.403083	-0.183174	0.679080	-0.500478	0.834995
6	0.744971	-0.339591	0.973977	0.935396	-0.692947	0.686966
7	0.079988	0.958470	-0.654606	-0.346182	0.006941	0.094851
8	0.011785	1.332368	-1.768900	-0.040575	0.006941	-0.102521
9	-0.124622	1.014907	-1.426040	-0.316607	0.164415	-0.250549
10	-2.068418	-1.214372	0.759690	-1.953083	2.404055	-2.059790
11	-2.341231	-0.988622	0.202543	-2.298123	2.334067	-2.372295
12	0.608564	-0.374865	0.888262	0.925537	-0.675450	0.522490
13	0.642666	0.161291	0.373973	0.876246	-0.605461	0.621175
14	0.097039	0.097799	1.145407	0.156591	-0.727941	0.094851
15	0.131140	0.972579	1.402551	-0.178591	-0.832924	0.061956
16	0.727920	-0.776981	1.402551	0.688938	-0.745438	0.769204
17	0.864326	-0.381919	0.545402	0.748088	-0.622958	0.818547
18	0.131140	0.224783	-0.740321	-0.198308	-0.098043	0.061956
19	-2.051367	-0.480685	0.545402	-2.357273	2.386558	-2.059790
20	0.796123	-1.023895	1.573981	0.935396	-0.762936	0.785652
21	0.028835	0.253002	-0.826036	-0.257457	-0.028054	-0.135416
22	0.796123	-1.313137	0.673975	0.659364	-0.640455	0.818547
23	0.267547	0.351767	-0.097459	-0.089866	-0.343003	0.127746
24	-1.863808	-0.311373	1.016834	-2.150249	2.456547	-1.862418
25	0.710869	-0.057404	0.116828	0.748088	-0.552969	0.637623
26	0.165242	0.203619	1.402551	0.067866	-0.797930	0.111299
27	0.779072	-1.009786	0.631117	0.757946	-0.640455	0.736309
28	0.079988	0.767994	-0.397461	-0.257457	-0.168031	0.029060
29	0.608564	-1.073278	0.631117	0.856529	-0.640455	0.736309

	0	1	2	3	4	5
...	...	...	...	...	...	...
1291	0.352801	1.431133	-0.097459	0.146733	-0.430489	0.390908
1292	0.881377	-0.551232	0.288258	0.797380	-0.587964	0.884338
1293	-1.897909	0.521080	-0.397461	-2.229115	2.229083	-1.829523
1294	0.762021	-0.642943	-0.568891	0.817096	-0.465483	0.802099
1295	0.864326	-0.360755	-0.054602	0.817096	-0.535472	0.950128
1296	0.625615	-0.043295	-0.654606	0.698797	-0.447986	0.752757
1297	0.540361	-0.424247	-0.354604	0.817096	-0.500478	0.506042
1298	0.199344	0.789158	-0.954608	-0.277174	0.024438	0.160642
1299	0.830225	-0.156170	0.373973	0.817096	-0.605461	0.818547
1300	0.898428	-0.431302	0.202543	0.876246	-0.587964	0.917233
1301	0.847276	0.097799	0.931119	0.895962	-0.675450	0.851442
1302	-0.022317	1.254766	-1.383183	-0.523631	0.409376	-0.069625
1303	0.079988	2.108383	-1.211753	-0.543348	0.409376	-0.020283
1304	0.148191	1.960235	-1.897472	-0.405332	0.339387	0.078403
1305	-2.153672	-0.734653	-0.268889	-2.238973	2.264078	-2.142028
1306	0.744971	-0.903966	-0.954608	0.403048	-0.343003	0.769204
1307	0.932530	-0.967458	-0.140317	0.708655	-0.517975	0.999471
1308	-1.880858	-0.699380	0.331115	-2.051666	2.334067	-1.845971
1309	0.216395	0.612791	-1.040323	-0.237741	0.006941	0.193537
1310	0.233445	0.485806	-0.868893	-0.227882	-0.045551	0.193537
1311	0.352801	0.923197	-0.783179	0.018575	-0.220523	0.308670
1312	-2.392384	-1.242590	-0.954608	-2.110816	2.124100	-2.306504
1313	0.676767	-0.960403	0.416830	0.866388	-0.605461	0.686966
1314	0.284598	0.782103	-1.040323	0.018575	-0.168031	0.308670
1315	0.744971	-0.353701	0.031113	0.836813	-0.552969	0.752757
1316	0.898428	-0.748763	-0.268889	0.915679	-0.517975	0.917233
1317	0.864326	-0.720544	0.888262	0.728372	-0.675450	0.900785
1318	0.762021	-0.261990	0.802547	0.876246	-0.657953	0.736309
1319	0.813174	-0.614724	0.759690	0.876246	-0.657953	0.818547

	0	1	2	3	4	5
1320	0.659716	-0.586505	-0.183174	0.787521	-0.517975	0.637623

1321 rows × 6 columns

Determining Pitches Through Clustering

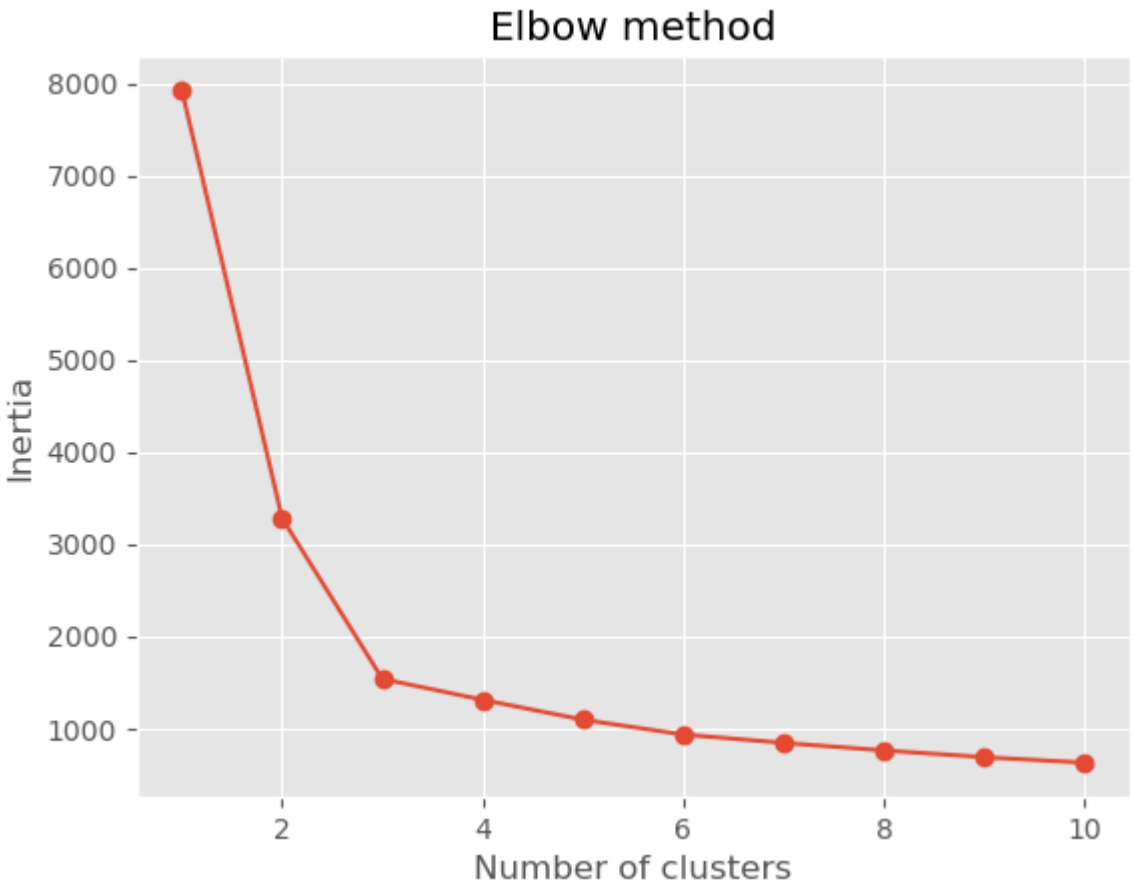
To be able to achieve the best predictions for strategy, first pitch movement data must be tracked and clustered to create organic pitch types because it has been estimated that 10% of all pitches are mislabeled by Statcast. For this, I will try a few clustering algorithms, including K-means, DBSCAN, and Gaussian Mixture Model.

To achieve the goal, I used a python package named Pybaseball to obtain all of the data I needed. This came in the form of 3 different called data sets, one which would be the most effective for clustering pitches together, one which gives general pitch data about a pitcher, and a third which does the same for the batter. First, I was able to estimate the number of unique pitches a pitcher may throw using the elbow method, which compares a sum of squares to values of K. This allowed me to look for an 'elbow', which is the location on the graph where the line begins running parallel to the x-axis. From there, I tested K-means, TSNE, Agglomerative, Gaussian Mixture, and DBSCAN clustering.

In [29]:
```
inertias = []
for i in range(1,11):
    kmeans = KMeans(n_clusters=i)
    kmeans.fit(scaledcluster)
    inertias.append(kmeans.inertia_)

plt.plot(range(1,11), inertias, marker='o')
plt.title('Elbow method')
plt.xlabel('Number of clusters')
plt.ylabel('Inertia')
plt.show()
```

```
C:\Users\erikw\anaconda3\lib\site-packages\sklearn\cluster\_kmeans.py:1412: FutureWa
rning: The default value of `n_init` will change from 10 to 'auto' in 1.4. Set the v
alue of `n_init` explicitly to suppress the warning
  super()._check_params_vs_input(X, default_n_init=10)
C:\Users\erikw\anaconda3\lib\site-packages\sklearn\cluster\_kmeans.py:1412: FutureWa
rning: The default value of `n_init` will change from 10 to 'auto' in 1.4. Set the v
alue of `n_init` explicitly to suppress the warning
  super()._check_params_vs_input(X, default_n_init=10)
C:\Users\erikw\anaconda3\lib\site-packages\sklearn\cluster\_kmeans.py:1412: FutureWa
rning: The default value of `n_init` will change from 10 to 'auto' in 1.4. Set the v
alue of `n_init` explicitly to suppress the warning
  super()._check_params_vs_input(X, default_n_init=10)
C:\Users\erikw\anaconda3\lib\site-packages\sklearn\cluster\_kmeans.py:1412: FutureWa
rning: The default value of `n_init` will change from 10 to 'auto' in 1.4. Set the v
alue of `n_init` explicitly to suppress the warning
  super()._check_params_vs_input(X, default_n_init=10)
C:\Users\erikw\anaconda3\lib\site-packages\sklearn\cluster\_kmeans.py:1412: FutureWa
rning: The default value of `n_init` will change from 10 to 'auto' in 1.4. Set the v
alue of `n_init` explicitly to suppress the warning
  super()._check_params_vs_input(X, default_n_init=10)
C:\Users\erikw\anaconda3\lib\site-packages\sklearn\cluster\_kmeans.py:1412: FutureWa
rning: The default value of `n_init` will change from 10 to 'auto' in 1.4. Set the v
alue of `n_init` explicitly to suppress the warning
  super()._check_params_vs_input(X, default_n_init=10)
C:\Users\erikw\anaconda3\lib\site-packages\sklearn\cluster\_kmeans.py:1412: FutureWa
rning: The default value of `n_init` will change from 10 to 'auto' in 1.4. Set the v
alue of `n_init` explicitly to suppress the warning
  super()._check_params_vs_input(X, default_n_init=10)
C:\Users\erikw\anaconda3\lib\site-packages\sklearn\cluster\_kmeans.py:1412: FutureWa
rning: The default value of `n_init` will change from 10 to 'auto' in 1.4. Set the v
alue of `n_init` explicitly to suppress the warning
  super()._check_params_vs_input(X, default_n_init=10)
C:\Users\erikw\anaconda3\lib\site-packages\sklearn\cluster\_kmeans.py:1412: FutureWa
rning: The default value of `n_init` will change from 10 to 'auto' in 1.4. Set the v
alue of `n_init` explicitly to suppress the warning
  super()._check_params_vs_input(X, default_n_init=10)
C:\Users\erikw\anaconda3\lib\site-packages\sklearn\cluster\_kmeans.py:1412: FutureWa
rning: The default value of `n_init` will change from 10 to 'auto' in 1.4. Set the v
alue of `n_init` explicitly to suppress the warning
  super()._check_params_vs_input(X, default_n_init=10)
```



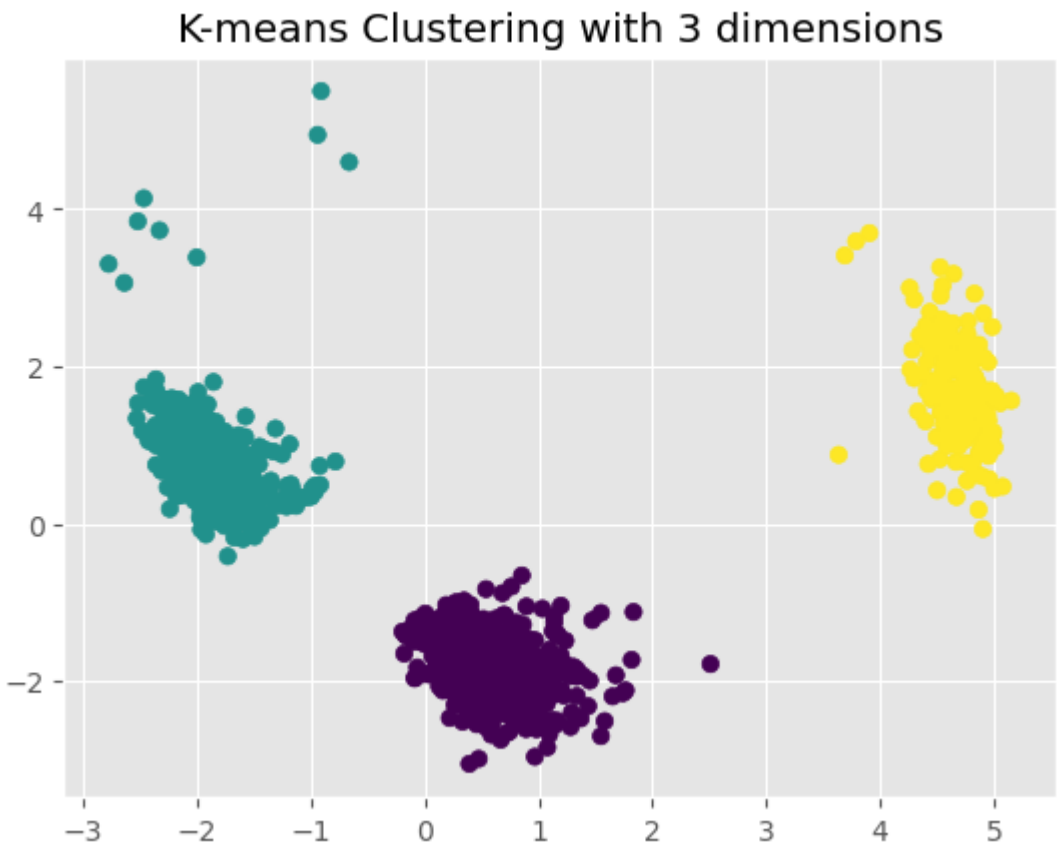
k-means compares values to centroids

In [30]:
```
kmeans = KMeans(n_clusters=3)
kmeans.fit(scaledcluster)
scaledcluster['kmeans_3'] = kmeans.labels_
```

```
C:\Users\erikw\anaconda3\lib\site-packages\sklearn\cluster\_kmeans.py:1412: FutureWa
rning: The default value of `n_init` will change from 10 to 'auto' in 1.4. Set the v
alue of `n_init` explicitly to suppress the warning
  super()._check_params_vs_input(X, default_n_init=10)
```

In [58]:
```
pca_num_components = 2
scaledcluster.columns = scaledcluster.columns.astype(str)
reduced_data = PCA(n_components=pca_num_components).fit_transform(scaledcluster)
results = pd.DataFrame(reduced_data,columns=['pca1','pca2'])
ss = silhouette_score(scaledcluster, kmeans.labels_)
print(ss)
plt.scatter(x="pca1", y="pca2", c=scaledcluster['kmeans_3'], data=results)
plt.title('K-means Clustering with 3 dimensions')
plt.show()
```

```
0.6874202532747692
```



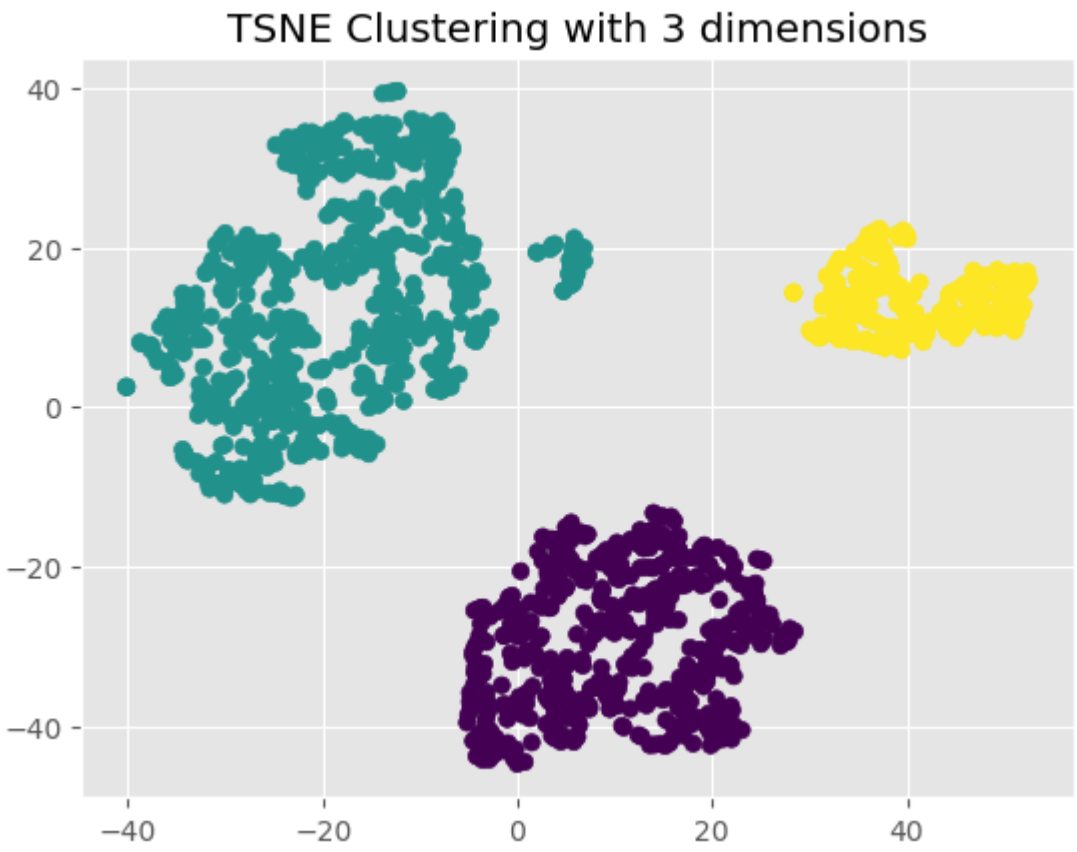
Alternative to PCA, nonlinear dimensionality reduction

In [59]:
```
tsne = TSNE(n_components=2,verbose=1, perplexity=30, n_iter=1000)
proj = tsne.fit_transform(scaledcluster.values)

tsneresults = pd.DataFrame(proj, columns=['tsne1','tsne2'])

ss = silhouette_score(proj, kmeans.labels_)
print(ss)
plt.scatter(x="tsne1", y="tsne2", c=scaledcluster['kmeans_3'], data=tsneresults)
plt.title('TSNE Clustering with 3 dimensions')
plt.show()
cluster_counts = scaledcluster['kmeans_3'].value_counts()
print(cluster_counts)
```

```
[t-SNE] Computing 91 nearest neighbors...
[t-SNE] Indexed 1321 samples in 0.004s...
[t-SNE] Computed neighbors for 1321 samples in 0.040s...
[t-SNE] Computed conditional probabilities for sample 1000 / 1321
[t-SNE] Computed conditional probabilities for sample 1321 / 1321
[t-SNE] Mean sigma: 0.237323
[t-SNE] KL divergence after 250 iterations with early exaggeration: 59.135628
[t-SNE] KL divergence after 1000 iterations: 0.634603
0.6552192
```




```
1    654
0    468
2    199
Name: kmeans_3, dtype: int64
```

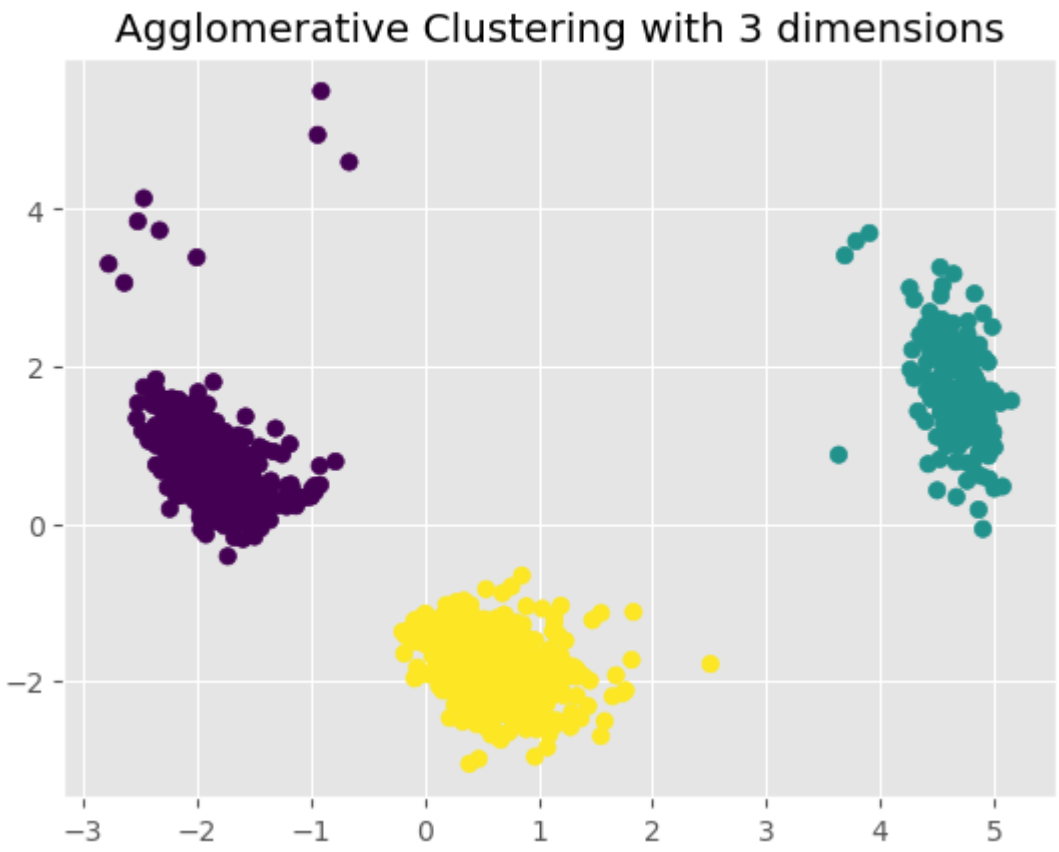
hierarchical clustering

In [33]:
```
hc = AgglomerativeClustering(n_clusters = 3)
hc.fit(scaledcluster.values)
scaledcluster['hc_3'] = hc.labels_
```

In [60]:
```
pca_num_components = 2

reduced_data = PCA(n_components=pca_num_components).fit_transform(scaledcluster.val
results = pd.DataFrame(reduced_data,columns=['pca1','pca2'])
ss = silhouette_score(scaledcluster.values, hc.labels_)
print(ss)
plt.scatter(x="pca1", y="pca2", c=scaledcluster['hc_3'], data=results)
plt.title('Agglomerative Clustering with 3 dimensions')
plt.show()
```

```
0.6874202532747692
```



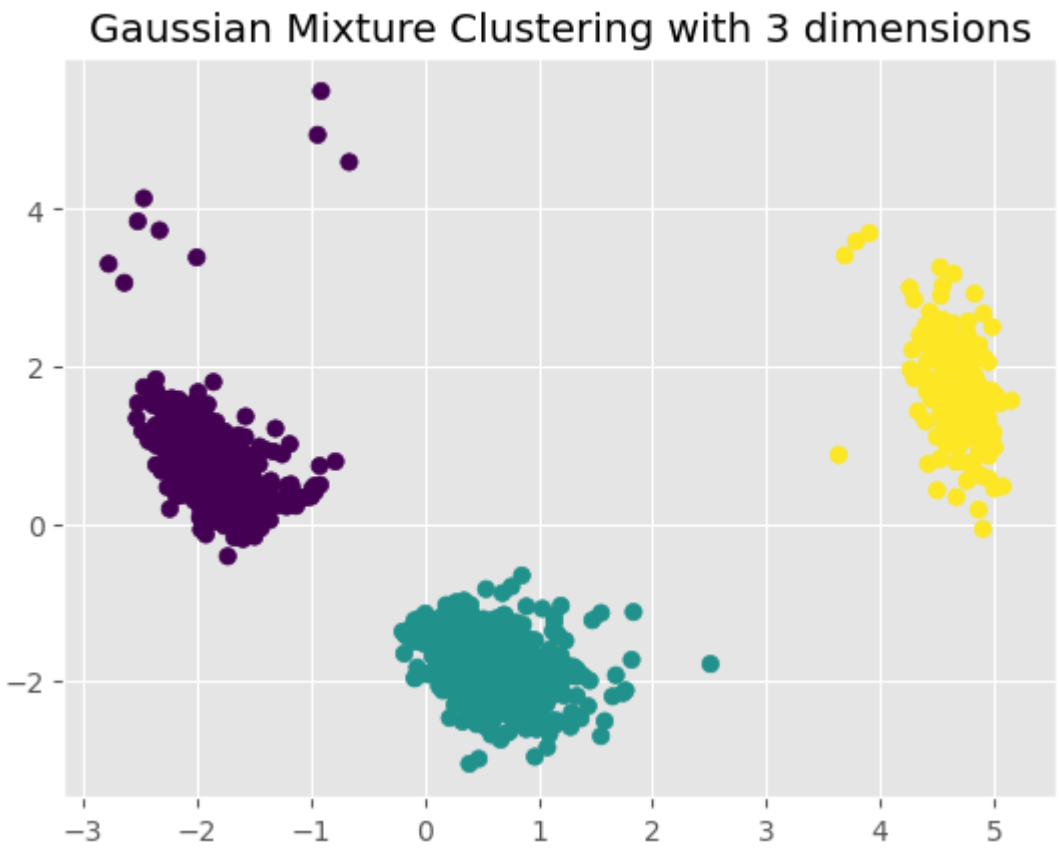
Similar to k-means, but handles oblongs better

In [35]:
```
gmm = GaussianMixture(n_components=3)
gmmlabels = gmm.fit_predict(scaledcluster.values)
scaledcluster['gmm_3'] = gmmlabels
```

In [61]:
```
pca_num_components = 2

reduced_data = PCA(n_components=pca_num_components).fit_transform(scaledcluster.val
results = pd.DataFrame(reduced_data,columns=['pca1','pca2'])
ss = silhouette_score(scaledcluster.values, gmmlabels)
print(ss)
plt.scatter(x="pca1", y="pca2", c=scaledcluster['gmm_3'], data=results)
plt.title('Gaussian Mixture Clustering with 3 dimensions')
plt.show()
```

```
0.6874202532747692
```



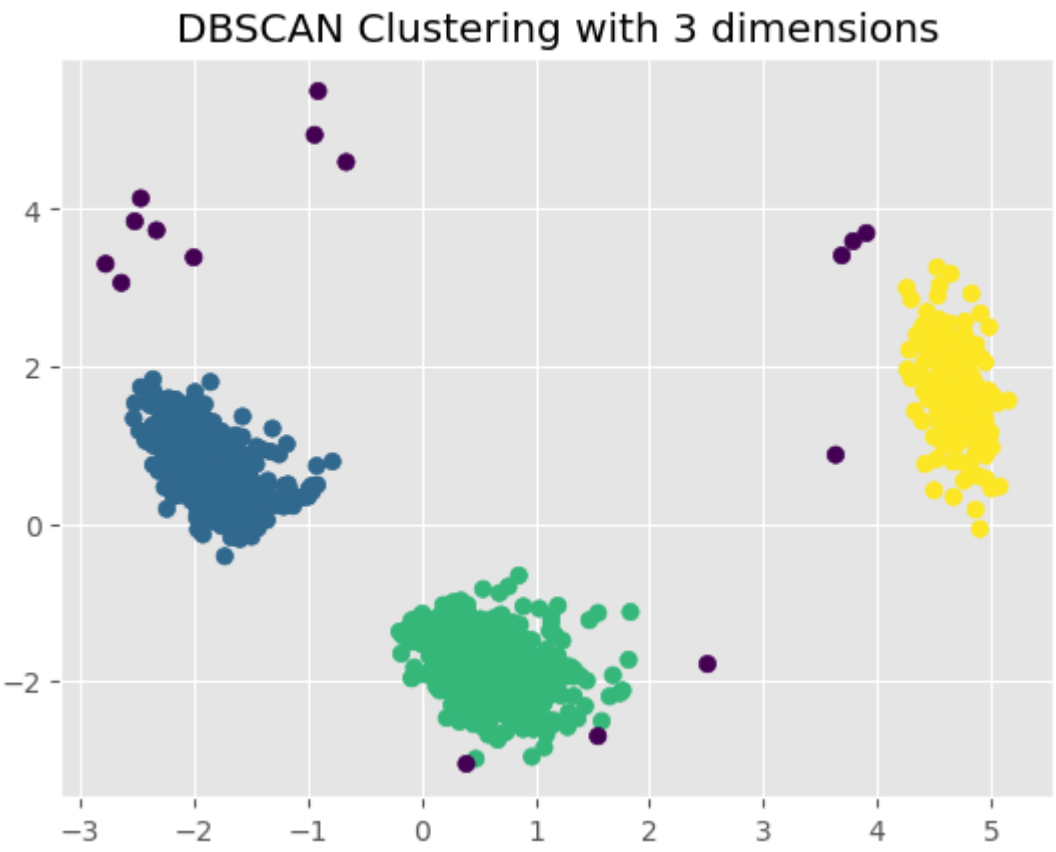
Finds dense regions surrounded by low density

In [37]:
```
dbscan = DBSCAN(eps=.8, min_samples=10)
dbscan.fit(scaledcluster.values)
scaledcluster['dbscan_3'] = dbscan.labels_
```

In [62]:
```
pca_num_components = 2

reduced_data = PCA(n_components=pca_num_components).fit_transform(scaledcluster.val
results = pd.DataFrame(reduced_data,columns=['pca1','pca2'])
ss = silhouette_score(scaledcluster.values, dbscan.labels_)
print(ss)
plt.scatter(x="pca1", y="pca2", c=scaledcluster['dbscan_3'], data=results)
plt.title('DBSCAN Clustering with 3 dimensions')
plt.show()
```

```
0.6888745383332268
```



Clustering Results

	K-means	TSNE	Agglomerative	GMM	DBSCAN
Silhouette Score	.687	.655	.687	.687	.689

Unfortunately, the testing and utilization of clustering was cut short due to source data errors with pybaseball. (If you'd like to see an example of this error, use the name 'Justin Verlander' as an example). But, in the testing that was done, I was first able to use what is called an elbow chart to make suggestions on the number of pitches clusters to expect on the data. This is done by looking for the point where the line begins to run parallel to the x-axis. After that, I was able to utilize 5 different methods of clustering and evaluate them using the silhouette score. The silhouette score is an ideal metric because it gives a score of just how tightly packed each point is with other points. Through the testing, each model showed promise, but unfortunately, due to the lack of data, this potential boost to the accuracy of the pitch strategy optimizer is unable to be used.

Pitch Strategy Prediction

Here, the functions dedicated to trying to obtain the optimal pitch strategy will be below.

The final model in answering the question of the most ideal strategy for baseball pitchers is a regression model. This is not the model I expected to be best for the task, but as I looked at the best way to weigh the failure of a batter and success of a pitcher, I quickly found that the results were going to be continuous. The idea of classifying a best pitch or worst pitch became illogical as each of those can vary so widely depending on the batter, pitcher, and overall situation.

In [39]:
```
#function defined to allow input of both player names, then have their id returned
def get_player_id(pitcher_name, batter_name):

    #split pitcher's name, then put split name into pybaseball's id lookup function
    first_name_pitcher, last_name_pitcher = pitcher_name.split(' ',1)
    first_name_pitcher = first_name_pitcher
    last_name_pitcher = last_name_pitcher
    get_id_pitcher = playerid_lookup(last_name_pitcher,first_name_pitcher)

    #split batter's name, then put split name into pybaseball's id lookup function
    first_name_batter, last_name_batter = batter_name.split(' ',1)
    first_name_batter = first_name_batter
    last_name_batter = last_name_batter
    get_id_batter = playerid_lookup(last_name_batter,first_name_batter)

    #the above is return in a bataframe, so grabbing just what I need
    #the .to_string(index=False) is used to get rid of duplicate index
    pitcher_id = get_id_pitcher['key_mlbam'].to_string(index=False)
    batter_id = get_id_batter['key_mlbam'].to_string(index=False)


    return pitcher_id, batter_id
```

Data Acquisition

In [40]:
```
#defined function to use pitcher_id to grab data that will be use
def get_pitcher_data(pitcher_id):

    statcast_data_pitcher = statcast_pitcher('2018-07-01', '2022-10-31', player_id
    #take data and select desired columns
    pitcher_data = statcast_data_pitcher[['pitch_type', 'description', 'stand', 'p_
    newpredictiondatapitcher = pitcher_data.copy()

    #only include pitch_type's that there are at least 100 of
    newpredictiondatapitcher = newpredictiondatapitcher[newpredictiondatapitcher.gr

    ### Key piece- create new column that is a sum of delta_run_exp based on what h
    ### This is to create an expected run variance for a pitch situation
    groupbypitcher = newpredictiondatapitcher.groupby(['zone','pitch_type','balls',
    sorted_pitcher_data = pd.merge(newpredictiondatapitcher,groupbypitcher, how='le

    return sorted_pitcher_data
```

In [41]:
```
# mimics get_pitcher_data
def get_batter_data(batter_id):

    statcast_data_batter = statcast_batter('2018-07-01', '2022-10-31', player_id =
    batter_data = statcast_data_batter[['batter','description','zone','pitch_type',
    newpredictiondatabatter = batter_data.copy()
    groupbybatter = newpredictiondatabatter.groupby(['zone','pitch_type','balls','s
    sorted_batter_data = pd.merge(newpredictiondatabatter,groupbybatter, how='left'

    return sorted_batter_data
```

Data Preperation

In [42]:
```
#most data cleaning in this function
def data_prep(sorted_pitcher_data, sorted_batter_data):


    # unique = sorted_pitcher_data.pitch_type.unique()

    #the next four lines take the most common listed hand thrown with for a pitcher
    #batter, then compare them to only keep data for the correct throwing hand/ bat
    most_common_p_throws_pitcher = sorted_pitcher_data['p_throws'].mode().iloc[0]
    sorted_batter_data = sorted_batter_data[sorted_batter_data['p_throws'] == most_
    most_common_stand_batter = sorted_batter_data['stand'].mode().iloc[0]
    sorted_pitcher_data = sorted_pitcher_data[sorted_pitcher_data['stand'] == most_

    #drop pitch_type values that are null
    sorted_pitcher_data = sorted_pitcher_data.dropna(subset=['pitch_type'])
    sorted_batter_data = sorted_batter_data.dropna(subset=['pitch_type'])

    # As seen in data exploration, batters face more pitches than most pitchers thr
    # they're seen in both pitcher and batter data as we'll never suggest a pitch t
    i = set(sorted_pitcher_data['pitch_type']) & set(sorted_batter_data['pitch_type
    combined_data = pd.concat([sorted_pitcher_data[sorted_pitcher_data['pitch_type'




    #replace null values with 0, reset the index
    combined_data.replace(np.nan,0)
    combined_data = combined_data.fillna(0)
    combined_data = combined_data.reset_index(drop=True)

    #drop columns that aren't needed anymore
    combined_data.drop(columns=['description', 'p_throws','stand'], inplace=True)

    #we will need pitch_type to be numerical
    combined_data.pitch_type = pd.Categorical(combined_data.pitch_type)
    combined_data['pitch_code'] = combined_data.pitch_type.cat.codes

    combined_data['previous_pitch'] = None
    for index, row in combined_data.iterrows():
        if row['pitch_number'] > 1:
            previous_row = combined_data.loc[
```

```
                (combined_data['at_bat_number'] == row['at_bat_number']) &
                (combined_data['pitch_number'] == row['pitch_number'] - 1)
            ]
            if not previous_row.empty:
                combined_data.at[index, 'previous_pitch'] = previous_row['pitch_cod

    combined_data['previous_pitch'] = combined_data['previous_pitch'].fillna(9)
    print(combined_data['previous_pitch'].value_counts())
    #map pitch_type so that when the numerical value is used later, we can call pit
    pitch_mapping = dict(zip(combined_data['pitch_code'], combined_data['pitch_type

    'at_bat_number', 'pitch_number',

    #currently these columns have player ids in them if a player is on base, I just
    combined_data['on_1b'] = combined_data['on_1b'].where(combined_data['on_1b'] <=
    combined_data['on_2b'] = combined_data['on_2b'].where(combined_data['on_2b'] <=
    combined_data['on_3b'] = combined_data['on_3b'].where(combined_data['on_3b'] <=

    #drop pitch_type
    combined_data.drop(columns=['pitch_type'], inplace=True)

    #convert values to integers
    combined_data['batter'] = combined_data['batter'].astype(int)
    combined_data['zone'] = combined_data['zone'].astype(int)
    combined_data['on_3b'] = combined_data['on_3b'].astype(int)
    combined_data['on_2b'] = combined_data['on_2b'].astype(int)
    combined_data['on_1b'] = combined_data['on_1b'].astype(int)
    combined_data['inning'] = combined_data['inning'].astype(int)
    combined_data['pitcher'] = combined_data['pitcher'].astype(int)

    ### Multiply by 1000 because I need integers and delta_run_exp is a decimal
    combined_data = combined_data.multiply(1000)

    #now convert these to int
    combined_data['delta_run_exp_x'] = combined_data['delta_run_exp_x'].astype(int)
    combined_data['delta_run_exp_y'] = combined_data['delta_run_exp_y'].astype(int)

    return combined_data, pitch_mapping
```

Data Preprocessing

In [43]:

```
#defining a function to encode
def data_preprocessing(combined_data):

    #label encoding values that are categorical
    label_encoder = LabelEncoder()
    #ordinal encoding values that have an order
    ordinal_encoder = OrdinalEncoder()
    combined_data['pitch_type_encoded'] = label_encoder.fit_transform(combined_data
    combined_data['previous_pitch_encoded'] = label_encoder.fit_transform(combined_
    combined_data['zone_encoded'] = label_encoder.fit_transform(combined_data['zone
    combined_data['on_3b_encoded'] = label_encoder.fit_transform(combined_data['on_
    combined_data['on_2b_encoded'] = label_encoder.fit_transform(combined_data['on_
    combined_data['on_1b_encoded'] = label_encoder.fit_transform(combined_data['on_
    combined_data['inning_encoded'] = ordinal_encoder.fit_transform(combined_data[[
```

```
    combined_data['balls_encoded'] = ordinal_encoder.fit_transform(combined_data[['
    combined_data['strikes_encoded'] = ordinal_encoder.fit_transform(combined_data[
    combined_data['outs_encoded'] = ordinal_encoder.fit_transform(combined_data[['o

    #defining features to be used later and the ultimate target
    features = ['on_3b_encoded', 'on_2b_encoded','on_1b_encoded','inning_encoded','
    target = 'delta_run_exp_y'

    return features, target, combined_data, ordinal_encoder
```

Model Exploration

I chose to test multiple models to see if anything was standing out. For the best performing models, I used GridSearchCV to optimize the parameters.

In [108…
```
#training model through randomforestregressor
def model_train(features, target, combined_data):

    #defining X, y and splitting data into training and test data
    X = combined_data[features]
    y = combined_data[target]
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random

    #training the model
    model = RandomForestRegressor(n_estimators = 200, max_features = 1.0, max_depth
    model.fit(X_train, y_train)

    #model score is essentially r^2
    print(model.score(X_test,y_test))


    return features, target, model, y_test, X_test
```

In [109…
```
#testing xgbregressor
def model_train1(features, target, combined_data):


    X = combined_data[features]
    y = combined_data[target]
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random


    model1 = xgb.XGBRegressor()
    model1.fit(X_train, y_train)
    print(model1.score(X_test,y_test))


    return features, target, model1, y_test, X_test
```

In [110…
```
# testing an ANN
def model_train2(features, target, combined_data):

    X = combined_data[features]
```

```
    y = combined_data[target]
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random

    #tested many different hidden layers
    model2 = Sequential()
    model2.add(Dense(units=128, activation='relu', input_dim=X_train.shape[1]))
    model2.add(Dense(units=64, activation='relu'))
    model2.add(Dense(units=32, activation='softmax'))
    model2.add(Dense(units=1))  # Output layer

    #adam optimizer
    opt = Adam(learning_rate=0.01)

    #compile and train, put verbose at 0 for this version to limit output
    model2.compile(loss='mean_squared_error', optimizer=opt)
    model2.fit(X_train, y_train, epochs=60, batch_size=64, verbose=0)



    return features, target, model2, y_test, X_test
```

In [111…

```
# testing linear regression
def model_train3(features, target, combined_data):

    X = combined_data[features]
    y = combined_data[target]
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random

    #create model and train
    model3 = LinearRegression()
    model3.fit(X_train, y_train)
    print(model3.score(X_test,y_test))


    return features, target, model3, y_test, X_test
```

In [112…

```
#testing lasso regression, values are essentially pulled towards mean
def model_train4(features, target, combined_data):

    X = combined_data[features]
    y = combined_data[target]
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random

    #alpha is a penalty, limit overfitting
    model4 = Lasso(alpha=1.0)
    model4.fit(X_train, y_train)

    print(model4.score(X_test,y_test))


    return features, target, model4, y_test, X_test
```

In [113…

```
#alternative model to xgboost
def model_train5(features, target, combined_data):
```

```
    X = combined_data[features]
    y = combined_data[target]
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random

    #parameters chosen through gridsearchcv
    model5 = CatBoostRegressor(iterations = 500, learning_rate = .04, depth = 9, lo
    model5.fit(X_train, y_train)

    print(model5.score(X_test,y_test))


    return features, target, model5, y_test, X_test
```

In [114…

```
'''from sklearn.model_selection import GridSearchCV
    X = combined_data[features]
    y = combined_data[target]

    # Train-Test Split
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random
    parameters = {'depth'         : [7,8,9, 10,11,12],
                  'learning_rate' : [.02,.04,.1,.3,.5,.7,.9],
                   'iterations'    : [200,300,400,500,600]
                  }

    # Create the CatBoost Regression model
    model5 = CatBoostRegressor()  # Adjust parameters as needed
    model5 = GridSearchCV(estimator=model5, param_grid = parameters, cv = 3, n_jobs
    model5.fit(X_train, y_train)
    # Train the model
    #model5.fit(X_train, y_train)

    print(" Results from Grid Search " )
    print("\n The best estimator across ALL searched params:\n",model5.best_estimat
    print("\n The best score across ALL searched params:\n",model5.best_score_)
    print("\n The best parameters across ALL searched params:\n",model5.best_params
    '''
```

Out[114…

```
'from sklearn.model_selection import GridSearchCV\n    X = combined_data[features]
\n    y = combined_data[target]\n    \n    # Train-Test Split\n    X_train, X_tes
t, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)\n    p
arameters = {\'depth\'         : [7,8,9, 10,11,12],\n                  \'learning_r
ate\' : [.02,.04,.1,.3,.5,.7,.9],\n                  \'iterations\'    : [200,300,
400,500,600]\n                  }\n    \n    # Create the CatBoost Regression model
\n    model5 = CatBoostRegressor()  # Adjust parameters as needed\n    model5 = Gr
idSearchCV(estimator=model5, param_grid = parameters, cv = 3, n_jobs=-1)\n    mode
l5.fit(X_train, y_train)\n    # Train the model\n    #model5.fit(X_train, y_train)
\n    \n    print(" Results from Grid Search " )\n    print("\n The best estimator
across ALL searched params:\n",model5.best_estimator_)\n    print("\n The best sco
re across ALL searched params:\n",model5.best_score_)\n    print("\n The best para
meters across ALL searched params:\n",model5.best_params_)\n    '
```

In [115…

```
#testing gradient boosting regressor
def model_train6(features, target, combined_data):

    X = combined_data[features]
```

```
    y = combined_data[target]
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random


    model6 = GradientBoostingRegressor(n_estimators=100, learning_rate=0.1, max_dep
    model6.fit(X_train, y_train)

    print(model6.score(X_test,y_test))


    return features, target, model6, y_test, X_test
```

Model Prediction

Below all of the models are used to make predictions and are evaluated. I determined that the best approach to evaluate both the success of the pitcher and the failure of the batter simultaneously was by using a shared statistic for the appropriate matchups. Because the idea of choosing a single best pitch on an island without feedback to suggest what would be expected in other situations felt so wrong, I decided to incorporate all possible combinations of situations and let the models determine how they all would perform. This allowed for the ideal heatmap as it gives information for all surrounding data, allowing the end-user to make a more informed decision

In [120…

```
#This is essentially the predictor function, for this, I kept all models
def get_best_zone_and_pitch_type(model, model1, model2, model3, model4, model5, mod
    # Create a dataframe with all possible combinations of zone and pitch_type

    #need number of pitches to make possible_combinations variable
    count = combined_data.pitch_type_encoded.nunique()

    #possible combinations is created to denote the number of values and particular
    possible_combinations = pd.DataFrame({

        'pitch_type_encoded': np.tile(np.arange(0, count), 13),
        'balls_encoded': [balls_encoded] * 13 * count,
        'strikes_encoded': [strikes_encoded] * 13 * count,
        'zone_encoded': np.repeat(np.arange(1, 14), count),
        'on_3b_encoded': [on_3b_encoded] * 13 * count,
        'on_2b_encoded': [on_2b_encoded] * 13 * count,
        'on_1b_encoded': [on_1b_encoded] * 13 * count,
        'inning_encoded': [inning_encoded] * 13 * count,
        'outs_encoded': [outs_encoded] * 13 * count,
        'previous_pitch_encoded': np.tile(np.arange(0, count), 13)
    })

    #Running each model for the possible combinations
    predictions = model.predict(possible_combinations[features])
    predictions1 = model1.predict(possible_combinations[features])
    predictions2 = model2.predict(possible_combinations[features])
    predictions3 = model3.predict(possible_combinations[features])
    predictions4 = model4.predict(possible_combinations[features])
    predictions5 = model5.predict(possible_combinations[features])
```

```
predictions6 = model6.predict(possible_combinations[features])

#making y_pred values for each model
y_pred = model.predict(X_test[features])
y_pred1 = model1.predict(X_test[features])
y_pred2 = model2.predict(X_test[features])
y_pred3 = model3.predict(X_test[features])
y_pred4 = model4.predict(X_test[features])
y_pred5 = model5.predict(X_test[features])
y_pred6 = model6.predict(X_test[features])

#divide each by 1000 to bring them back down to real numbers
y_test = y_test.div(1000)
y_pred = np.divide(y_pred, 1000)
y_pred1 = np.divide(y_pred1, 1000)
y_pred2 = np.divide(y_pred2, 1000)
y_pred3 = np.divide(y_pred3, 1000)
y_pred4 = np.divide(y_pred4, 1000)
y_pred5 = np.divide(y_pred5, 1000)
y_pred6 = np.divide(y_pred6, 1000)

#each of these sets are going to be used to help evaluate the accuracy of the r
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y_test, y_pred)

print(f"Mean Squared Error: {mse:.2f}")
print(f"Root Mean Squared Error: {rmse:.2f}")
print(f"Mean Absolute Error: {mae:.2f}")

mse1 = mean_squared_error(y_test, y_pred1)
rmse1 = np.sqrt(mse1)
mae1 = mean_absolute_error(y_test, y_pred1)

print(f"Mean Squared Error: {mse1:.2f}")
print(f"Root Mean Squared Error: {rmse1:.2f}")
print(f"Mean Absolute Error: {mae1:.2f}")

mse2 = mean_squared_error(y_test, y_pred2)
rmse2 = np.sqrt(mse2)
mae2 = mean_absolute_error(y_test, y_pred2)

print(f"Mean Squared Error: {mse2:.2f}")
print(f"Root Mean Squared Error: {rmse2:.2f}")
print(f"Mean Absolute Error: {mae2:.2f}")

mse3 = mean_squared_error(y_test, y_pred3)
rmse3 = np.sqrt(mse3)
mae3 = mean_absolute_error(y_test, y_pred3)

print(f"Mean Squared Error: {mse3:.2f}")
print(f"Root Mean Squared Error: {rmse3:.2f}")
print(f"Mean Absolute Error: {mae3:.2f}")

mse4 = mean_squared_error(y_test, y_pred4)
rmse4 = np.sqrt(mse4)
```

```
    mae4 = mean_absolute_error(y_test, y_pred4)

    print(f"Mean Squared Error: {mse4:.2f}")
    print(f"Root Mean Squared Error: {rmse4:.2f}")
    print(f"Mean Absolute Error: {mae4:.2f}")

    mse5 = mean_squared_error(y_test, y_pred5)
    rmse5 = np.sqrt(mse5)
    mae5 = mean_absolute_error(y_test, y_pred5)

    print(f"Mean Squared Error: {mse5:.2f}")
    print(f"Root Mean Squared Error: {rmse5:.2f}")
    print(f"Mean Absolute Error: {mae5:.2f}")

    mse6 = mean_squared_error(y_test, y_pred6)
    rmse6 = np.sqrt(mse6)
    mae6 = mean_absolute_error(y_test, y_pred6)

    print(f"Mean Squared Error: {mse6:.2f}")
    print(f"Root Mean Squared Error: {rmse6:.2f}")
    print(f"Mean Absolute Error: {mae6:.2f}")


    #get the best index by finding the minimum value of delta_run_exp_y
    best_idx = np.argmin(predictions)

    #get the best zone and pitch_type based on the index above
    best_zone = possible_combinations.loc[best_idx, 'zone_encoded']
    best_pitch_type_encoded = possible_combinations.loc[best_idx, 'pitch_type_encod

    #just for plot
    y_actual = y_test

    #scatter plot showing visually how well it has been predicted
    plt.figure(figsize=(8, 6))
    plt.scatter(y_actual, y_pred, color='blue', alpha=0.5)
    plt.title('Actual vs. Predicted Values')
    plt.xlabel('Actual Values')
    plt.ylabel('Predicted Values')
    plt.grid(True)
    plt.show()


    #get best by inverse_transforming the pitch type and then dividing by 1000, ret
    best_pitch_type = ordinal_encoder.inverse_transform(best_pitch_type_encoded.res
    best_pitch_type = int(best_pitch_type/1000)
    pitch_code = best_pitch_type
    pitch_type_associated = pitch_mapping[pitch_code]

    return predictions, best_zone, best_pitch_type, possible_combinations, pitch_ty
```

Main Function

This function is dedicated to calling all other defined functions. Overall creating all of these functions was done to allow for the code to be adaptable and reuseable.

In [117…

```
#main function that will take in form data on web app and then go through all the d
#can easily remove models, but for testing, I will leave them
def main(pitcher_name, batter_name, balls_encoded, strikes_encoded, outs_encoded, o
    pitcher_id, batter_id = get_player_id(pitcher_name, batter_name)
    sorted_pitcher_data = get_pitcher_data(pitcher_id)
    sorted_batter_data = get_batter_data(batter_id)
    combined_data, pitch_mapping = data_prep(sorted_pitcher_data, sorted_batter_dat
    features, target, combined_data, ordinal_encoder = data_preprocessing(combined_
    features, target, model, y_test, X_test = model_train(features, target, combine
    features, target, model1, y_test, X_test = model_train1(features, target, combi
    features, target, model2, y_test, X_test = model_train2(features, target, combi
    features, target, model3, y_test, X_test = model_train3(features, target, combi
    features, target, model4, y_test, X_test = model_train4(features, target, combi
    features, target, model5, y_test, X_test = model_train5(features, target, combi
    features, target, model6, y_test, X_test = model_train6(features, target, combi
    predictions, best_zone, best_pitch_type, possible_combinations, pitch_type_asso
    return predictions, best_zone, best_pitch_type, possible_combinations, pitch_ty
```

Test Parameters

This just allows for testing when the input isn't from a form on a web app

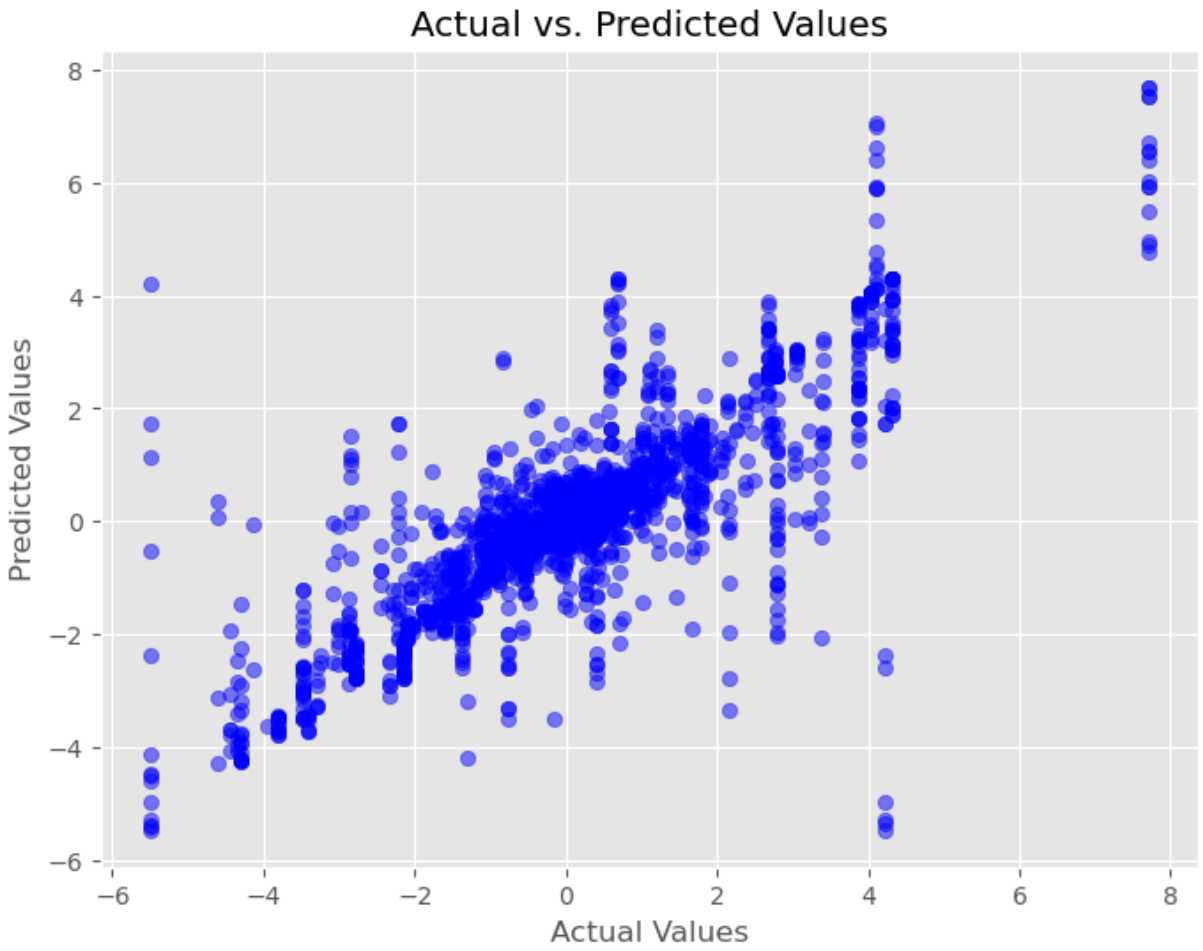
In [121…

```
# This is test usage. These values will come from the forms on the web app
#batter_id = 545361
#pitcher_id = 477132
pitcher_name = 'Justin Verlander'
batter_name = 'Nolan Arenado'
balls_encoded = 0
strikes_encoded = 0
inning_encoded = 2
outs_encoded = 1
on_3b_encoded = 0
on_2b_encoded = 0
on_1b_encoded = 1

#call main
predictions, best_zone, best_pitch_type, possible_combinations, pitch_type_associat

#output
print(f"The best zone for the pitcher is: {best_zone}")
print(f"The best pitch type for the pitcher is: {best_pitch_type} {pitch_type_assoc
```

```
Gathering Player Data
Gathering Player Data
0    4238
9    2513
1    2130
2     541
3      21
Name: previous_pitch, dtype: int64
0.6494962656220302
0.6679155199469882
0.1279287260965325
0.12791713585891518
0:      learn: 2004.3424089     total: 2.53ms   remaining: 1.26s
200:    learn: 1111.6123168     total: 375ms    remaining: 558ms
400:    learn: 996.6547926      total: 745ms    remaining: 184ms
499:    learn: 962.3539758      total: 925ms    remaining: 0us
0.6802954932587362
0.5488883293016968
2/2 [==============================] - 0s 15ms/step
60/60 [==============================] - 0s 786us/step
Mean Squared Error: 1.43
Root Mean Squared Error: 1.20
Mean Absolute Error: 0.70
Mean Squared Error: 1.36
Root Mean Squared Error: 1.16
Mean Absolute Error: 0.81
Mean Squared Error: 4.09
Root Mean Squared Error: 2.02
Mean Absolute Error: 1.48
Mean Squared Error: 3.56
Root Mean Squared Error: 1.89
Mean Absolute Error: 1.40
Mean Squared Error: 3.56
Root Mean Squared Error: 1.89
Mean Absolute Error: 1.40
Mean Squared Error: 1.31
Root Mean Squared Error: 1.14
Mean Absolute Error: 0.79
Mean Squared Error: 1.84
Root Mean Squared Error: 1.36
Mean Absolute Error: 1.02
```




```
The best zone for the pitcher is: 1
The best pitch type for the pitcher is: 2 FF
```

Heatmap

This is the primary visualization due to its common utilization in baseball visualization. I made the choice to only show the data for the pitches in the strikezone. The pitches outside the zone are never truly recommended by the algorithm, so this allows more focus on the prediction.

In [119…

```
predictions = pd.DataFrame(predictions)

#calculate number of pitch types
total_groups = len(predictions) // 13

heatmap_data_list = []
#split data which was previously iterated by pitch type first
#now data will iterate by zone to allow heatmap
for value_idx in range(best_pitch_type, 9 * total_groups, total_groups):
    group_values = predictions.iloc[value_idx, 0]
    heatmap_data_list.append(group_values)


heatmap_data = pd.Series(heatmap_data_list).values.reshape(3, 3)
```

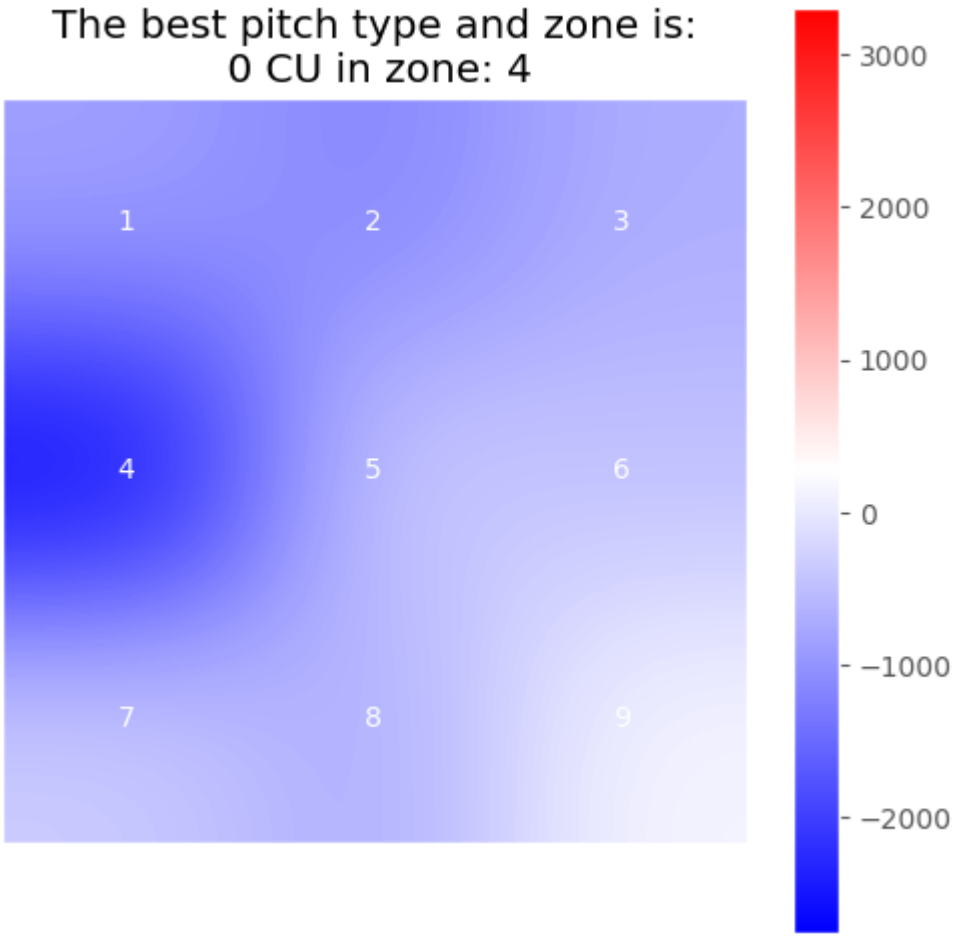
```
plt.figure(figsize=(6, 6))
plt.imshow(heatmap_data, cmap='bwr', interpolation='gaussian', vmin=predictions.min
plt.grid(True, color='gray', linewidth=0.5, alpha=0.5)

# Loop to display numbers in the middle of each zone
for i in range(3):
    for j in range(3):
        plt.text(j, i, str(i*3 + j + 1), ha='center', va='center', color='white')

plt.xticks([])
plt.yticks([])

plt.colorbar()
plt.title(f"The best pitch type and zone is:\n {best_pitch_type} {pitch_type_associ
plt.show()
```

```
C:\Users\erikw\AppData\Local\Temp\ipykernel_2532\799361738.py:34: MatplotlibDeprecat
ionWarning: Auto-removal of grids by pcolor() and pcolormesh() is deprecated since
3.5 and will be removed two minor releases later; please call grid(False) first.
  plt.colorbar()
```



Results

	Random Forest	XGBoost	ANN	Linear	Lasso	CatBoost	GradientBoost
Model Score	.925	.917		.123	.124	.924	.679
---	---	---	---	---	---	---	---
MSE	.41	.45	5.49	4.73	4.73	.41	1.73
---	---	---	---	---	---	---	---
RMSE	.64	.67	2.34	2.17	2.17	.64	1.32
---	---	---	---	---	---	---	---
MAE	.20	.32	1.59	1.57	1.57	.33	.99
---	---	---	---	---	---	---	---

Results Discussion

In the table above, along with the heatmap, scatter plot, and predictions, we see the results for the pitch optimization strategy. Starting with the models assessed, we see that I chose 7 different regression models that were all using the features 'on_3b_encoded', 'on_2b_encoded','on_1b_encoded','inning_encoded','outs_encoded', 'balls_encoded', 'strikes_encoded', 'zone_encoded', 'pitch_type_encoded','previous_pitch_encoded'.

These features were chosen because in the exploratory analysis, it was found that they all have low correlation to each other and each should have an impact on when, where, and what pitch is thrown. These will all help to lead to the target, which is the statistic delta_run_exp, which is an incredibly powerful statistic used to assess the expected runs each pitch will lead to. For the purpose of this project, this statistic is perfect because it has negative values for a hitter, suggesting his failure, and negative values for a pitcher, suggesting he is less likely to give up a run.

So, realizing this was the perfect target, I looked at potential models and decided to try each and compare results. For any results that are close, I did run a GradientSearchCV to try to find the best parameters, but on multiple occasions found that the default parameters were the most consistent.

As seen in the chart above, the random forest regressor scored well across all metrics. I was particularly focused on RMSE and MAE because I felt as though they complimented each other well. RMSE has much heavier penalties for outliers, which can be very important in a situation like baseball, but, the simplicity of mean average error also is nice to see.

Moving on to the predictions, the output is simply the zone and pitch type combination that is likely to have the best result for the pitcher. While this is the primary result, the path to achieving it is done through having the model predicting the target value for every pitch and zone combination, then, finding the one that favored the pitcher the most, and returning that. The heatmap is returned showing the range of expected success for a the pitcher and

hitter in each locaiton, which is important because in a realistic game, it is important to know where you'd like to 'miss'. Lastly, the scatter plot was created as a nice visual to quickly get the feel of how accurate the predictions have been.

Results Conclusion

Ultimately, I do feel as though my algorithm was able to do what I set out to do. With varying accuracy, but always showing at very least a strong correlation, it was able to show that the predicted values did match the test values. Baseball is a fickle sport, so I was never expecting to be able to predict this with 100% accuracy. Also, I absolutely expected some hitters or pitchers to be less predictable than others, so overall, I feel as though my results supported everything I did expect.

In [ ]: